

AI 와 함께하는 임베디드 개발

Vol.4 — Physical AI

라즈베리파이 · YOLOv8 · GPS · TC237 과 함께 만드는 자율
추적 RC 카

Toby · AI-Craft

이 책을 시작하며 — 우리는 어디까지 왔고, 어디로 가는가

Vol.1 에서 우리는 임베디드의 첫 발을 떼었습니다. TC237 보드를 손에 쥐고, MCU 의 세계가 어떻게 작동하는지 — 레지스터를 직접 만지는 것부터 인터럽트가 어떻게 흐르는지까지 — 처음으로 배웠습니다.

Vol.2 에서는 그 첫 발에 다리를 붙였습니다. ADC 로 아날로그 세계를 읽고, PWM 으로 모터를 돌리고, UART·SPI·CAN 으로 다른 칩들과 대화하는 법을 익혔습니다. Flash 에 데이터를 저장하는 일까지. MCU 의 손과 발이 되어주는 페리페럴들을 하나씩 정복했습니다.

Vol.3 에서는 그 손과 발을 모아 비로소 "움직이는 것"을 만들었습니다. 모터를 돌리고, HC-12 로 무선 조종하고, 거리 센서로 장애물을 인식하고, MPU9250 자이로로 똑바로 달리게 했습니다. RC 카가 우리 앞에서 동작했습니다.

자율주행이라 부르기에 소박했지만, MCU 한 대로 할 수 있는 시스템의 상한선에 가까이 갔습니다.

그리고 우리는 한 가지를 깨달았습니다 — MCU 만으로는 닿을 수 없는 영역이 있다는 것을. 카메라로 세상을 보고, AI 로 의미를 추출하고, 위성으로 위치를 알아내는 일은 TC237 의 영토 밖에 있습니다. 200MHz 의 TriCore 도, 4MB 의 Flash 도, 부동소수점 유닛도 — YOLO 같은 신경망 앞에서는 부족합니다.

Vol.4 는 그 경계를 넘어섭니다.

우리는 라즈베리파이를 RC 카의 "눈과 뇌"로 엮습니다. Pi Camera 로 시각 정보를 받고, YOLOv8 로 사물을 인식하고, 우리집 고양이만 알아보도록 직접 데이터를 모아 모델을 학습시킵니다. GPS 로 위치를 기록하고, 마지막으로 TC237 과 손을 잡아 — 고양이를 따라다니는 자율 차량을 완성합니다.

이것이 Physical AI 입니다. 화면 속에 갇혀 있던 인공지능이 바퀴를 달고 세상으로 나오는 순간입니다. ChatGPT 가 텍스트로 답하고 Midjourney 가 그림을 그리는 디지털 AI 와 달리, Physical AI 는 카메라로 세상을 보고, 모터로 세상을 바꿉니다. 자율주행차, 산업용 로봇, 가정용 로봇청소기, 농업 드론 — 우리 시대의 거의 모든 "움직이는 AI"가 같은 구조로 동작합니다. 우리가 만들 RC 카는 그 구조의 가장 단순한 시제품입니다.

그리고 이 모든 여정에서, 우리는 혼자가 아닙니다. AI 는 더 이상 우리가 만드는

시스템 안에만 있지 않습니다. AI 는 우리와 함께 시스템을 설계하는 동료입니다. 부품을 고르고, 아키텍처를 그리고, 코드를 짜고, 결과를 해석하는 모든 순간에 우리는 AI 에게 묻고, AI 는 답하고, 우리는 그것을 검증합니다.

Vol.1 부터 Vol.3 까지 우리는 AI 에게 코드를 부탁했습니다. Vol.4 부터는 AI 와 함께 시스템을 설계합니다.

이 책은 단순한 AI 활용 기법서가 아닙니다. 임베디드 개발자가 — 양산차 ECU 를 다루던 손으로 — 어떻게 AI 시대를 살아갈 것인가에 대한 한 가지 답입니다. 결정은 사람이 하고, 책임도 사람이 집니다. AI 는 우리 옆에 앉아 빠르게 초안을 만들고 답을 검토해 주는 좋은 후배 엔지니어일 뿐입니다. 그 균형이 무너지면 좋은 시스템은 만들어지지 않습니다.

자, 마지막 여정을 시작하겠습니다. Vol.1 에서 우리가 처음 만났던 그 TC237 보드가 — 이제 라즈베리파이의 명령을 받아 자율적으로 움직이는 차량의 일부가 되는 모습을 — 함께 보시지요.

— Toby

Chapter 1. 왜 라즈베리파이를 더하는가? — MCU 의 한계와 SoC 의 역할

1.1 Vol.3 까지의 RC 카, 무엇이 부족했나

Vol.3 에서 우리는 TC237 한 대로 꽤 많은 일을 해냈습니다. UART 로 들어오는 명령에 따라 모터를 돌렸고, HC-12 로 무선 조종 채널을 만들었고, 적외선 거리 센서로 장애물을 회피했고, MPU9250 자이로로 직진 경로를 유지했습니다. 마지막으로 yaw 보정 루프까지 넣어, 외란이 있어도 차가 똑바로 달리도록 만들었습니다.

자율주행이라는 단어를 쓰기에는 소박했지만, 임베디드 개발자가 MCU 한 대로 만들 수 있는 시스템의 상한선에 가까이 갔습니다. Vol.3 가 "AURIX 의 영토 안에서 RC 카가 할 수 있는 모든 것"을 다뤘다고 해도 과언이 아닙니다.

그런데 만약 이 RC 카에게 다음과 같은 임무를 준다면 어떨까요?

- 거실에서 우리집 고양이를 찾아 따라가라.
- 실시간으로 카메라 영상을 분석해, 무엇이 있는지 알아내라.
- 현재 위치를 GPS 로 기록하라.
- 학습된 AI 모델로 객체를 인식하라.
- 나중에 "오늘 어느 길로 다녔는지"를 지도 위에 그려라.

TC237 은 이 자리에서 손을 듭니다 — "저는 못합니다." 그것도 "조금만 노력하면 됩니다"가 아니라 "근본적으로 못합니다"입니다. 그 이유를 짚어보겠습니다.

1.2 카메라·AI·GPS 는 왜 TC237 으로 안 되는가 — 연산량·메모리·OS 의 벽

MCU 와 카메라·AI·GPS 의 궁합이 맞지 않는 이유는 크게 세 가지입니다. 하나는 연산량의 벽, 둘은 메모리의 벽, 셋은 OS 와 생태계의 벽입니다.

연산량의 벽

YOLOv8 같은 객체 인식 모델은 한 프레임을 처리하는 데 수십억 번의 곱셈·덧셈이 필요합니다. 더 정확하게 말하면, YOLOv8n(nano, 가장 작은 버전) 모델은 약 8.7 GFLOPs 입니다 — 한 프레임당 87 억 번의 부동소수점 연산입니다.

TC237 의 TriCore 는 200MHz 로 동작하고 부동소수점 연산도 가능하지만, 본래

실시간 제어를 위해 설계된 칩입니다. 명령어 처리량(throughput)으로 따지면
단정밀도 부동소수점 곱셈을 초당 약 1~2 억 번 정도 할 수 있습니다. 단순
계산으로도 한 프레임을 분 단위로 계산해야 한다는 뜻입니다.

같은 모델을 라즈베리파이 4(쿼드코어 1.8GHz, NEON SIMD 지원)에서 돌리면 초당
3~5 프레임이 나옵니다. 라즈베리파이 5(2.4GHz)에서는 8~12 프레임입니다.
같은 작업이지만 "분 단위" 와 "수 프레임" 의 차이입니다. 이걸 노력으로 메울 수
있는 간격이 아닙니다.

임베디드 개발자의 직관:

MCU 와 SoC 의 차이는 단순히 '클럭 속도'가 아닙니다. NEON SIMD, GPU, 캐시 구조,
메모리 대역폭 — 모든 것이 다릅니다. 차 한 대로 짐을 옮기는 것과 트럭으로 옮기는
것의 차이라고 보면 됩니다.

메모리의 벽

두 번째 벽은 메모리입니다. YOLOv8n 모델 파일 자체가 약 6.2 MB 입니다. TC237
의 내장 Flash 는 4 MB, RAM 은 수백 KB 수준입니다. 모델을 올려놓을 공간조차
없습니다.

물론 외부 SPI Flash 나 SD 카드를 붙여 모델을 저장할 수도 있습니다. 그러나 추론
도중에 모델의 가중치를 RAM 에 일부씩 로드하면서 계산하는 것은 — 가능하지만
극도로 비효율적입니다. 캐시 미스가 계속 일어나고, 결과적으로 "분 단위로 한
프레임"이 "시간 단위로 한 프레임"이 됩니다.

이 정도 설계 복잡도라면 차라리 처음부터 SoC 를 쓰는 것이 모든 면에서
합리적입니다. 임베디드 개발의 첫 번째 원칙 중 하나가 "적절한 도구를 적절한
자리에 쓰는 것"입니다. MCU 에 AI 모델을 욱여넣는 것은 그 원칙에 어긋납니다.

OS 와 생태계의 벽

세 번째 벽은 더 결정적입니다. OpenCV 로 영상을 받고, PyTorch 로 모델을
추론하고, NMEA 라이브러리로 GPS 를 파싱하는 — 이 모든 작업은 Linux 와
Python 생태계 위에서 굴러갑니다. RTOS 도 좋고 베어메탈도 좋지만, 비전과 AI 는
풍부한 라이브러리가 있는 OS 의 영토입니다.

물론 모든 것을 직접 짤 수도 있습니다. JPEG 디코더, 행렬 곱셈 라이브러리, NMEA
파서 — 다 손으로 만들 수 있습니다. 그러나 그것은 "바퀴를 재발명하는 일"입니다.

우리에게는 그 시간에 더 가치 있는 일이 있습니다. 우리집 고양이를 어떻게 인식할지, RC 카가 어떻게 안전하게 움직일지 — 그런 "우리 시스템 고유의 문제"에 시간을 쓰는 것이 좋습니다.

Linux 위에서 굴러가는 풍부한 라이브러리 생태계는 — 누군가가 이미 "바퀴를 만들어 놓은 것" 입니다. 그 위에 올라타면 됩니다.

그러면 답은 분명합니다

MCU 의 영토를 넘는 작업이 필요할 때, 답은 명확합니다 — MCU 옆에 SoC 를 두는 것입니다. 라즈베리파이는 학생도 살 수 있는 가격(8~14 만원)에, Linux 와 Python 생태계를 그대로 누릴 수 있는, 임베디드 학습용 SoC 의 표준입니다.

1.3 실제 자동차 **ADAS** 구조 — 카메라 **ECU** 와 차량 제어 **ECU** 의 분리

이 분리는 우리가 처음 발견한 패턴이 아닙니다. 실제 양산 차량의 ADAS(Advanced Driver Assistance System) 시스템은 이미 오래전부터 같은 방식으로 설계되어 있습니다.

Tesla 의 FSD 컴퓨터, Mobileye 의 EyeQ 시리즈, NVIDIA 의 Orin, Qualcomm 의 Snapdragon Ride — 이런 칩들은 카메라 영상을 받아 차선·차량·보행자·표지판을 인식합니다. 이들은 모두 강력한 GPU 또는 NPU 를 갖춘 SoC 이며, Linux 또는 QNX 같은 OS 위에서 동작합니다.

그러나 이 SoC 는 직접 브레이크를 밟거나 핸들을 돌리지 않습니다. 인식 결과를 차량 제어 ECU(많은 경우 Infineon AURIX 같은 안전 등급 MCU)에 전달하고, 실제 차량 제어는 그쪽에서 결정론적으로 수행됩니다.

이유는 명확합니다 — 인공지능은 빠르지만 비결정적이고, MCU 는 느리지만 결정적이기 때문입니다.

AI 가 "앞에 차가 있다"고 판단하더라도, 실제로 브레이크를 얼마나 어떻게 밟을지는 안전성이 검증된 MCU 가 결정해야 합니다. 자동차 산업의 ISO 26262 ASIL-D 같은 기능안전 요구사항은 SoC 보다 MCU 가 충족하기 쉽습니다. 신경망은 "왜 그렇게 판단했는지"를 설명하기 어렵지만, MCU 의 제어 로직은 모든 분기마다 "왜 그렇게 동작하는지"를 설명할 수 있습니다.

임베디드 개발자에게 의미:

Vol.4 에서 우리가 만드는 라즈베리파이 + TC237 구조는 장난감이 아닙니다. 양산차 ADAS 의 축소판입니다. AI 는 보고 판단하고, MCU 는 결정론적으로 움직입니다. 이 구조가 양산차에서 검증된 패턴이라는 점은, 여러분이 만드는 시스템에도 신뢰를 더해 줍니다.

덧붙이자면, 자동차뿐 아니라 산업용 로봇·드론·자율 농기계도 같은 분리 구조를 따릅니다. 인지(Perception) 레이어는 SoC 가, 제어(Control) 레이어는 MCU 가 — 이 분업이 "임베디드 + AI" 시스템의 사실상 표준입니다. Vol.4 의 RC 카는 그 표준을 가장 단순한 형태로 구현한 것입니다.

1.4 Vol.4 RC 카의 시스템 구성도와 역할 분담

우리가 Vol.4 에서 완성할 RC 카의 전체 구조는 다음과 같습니다.

[라즈베리파이 4 또는 5]

```
|
├── Pi Camera (CSI)      → 카메라 영상 수신
├── GPS NEO-M8N (UART)   → 위도·경도 수신
├── USB-UART 어댑터      → TC237 과 명령 송수신
|
| ▼ 명령 패킷 (UART, 9600bps)
```

[TC237 보드]

```
|
├── L298N x 2 (PWM 출력) → DC 모터 4 개 (전/후 좌우)
├── IR 거리 센서 (ADC)    → 장애물 회피 (안전 계층)
├── 초음파 센서 (GPIO)   → 근거리 정밀 측정
├── MPU9250 (QSPI3)      → 자이로/가속도 (자세 보정)
├── 배터리 모니터 (ADC)  → 배터리 전압 감시
```

역할 분담은 분명합니다.

- 라즈베리파이 = 인지(Perception). 카메라로 보고, AI 로 인식하고, GPS 로 위치를 안다. "무엇을, 어디로" 같이 결정한다.
- TC237 = 제어(Control). 라즈베리파이의 명령을 받아 안전하게 모터를 돌린다. 센서로 충돌을 막고, 자세를 유지한다. "어떻게" 가야 안전한지 결정한다.

이 분업의 좋은 점은 — Vol.3 에서 만든 TC237 펌웨어를 거의 그대로 재사용한다는

것입니다. 다르다면 "UART 명령의 출처가 PC가 아니라 라즈베리파이로 바뀐다"는 점뿐입니다. Vol.3의 명령 프로토콜(FORWARD, STOP, STEER 같은 것들)이 그대로 살아 있습니다.

Vol.1부터 Vol.3까지 쌓아온 코드와 펌웨어가 Vol.4의 토대가 됩니다. 그 위에 "카메라와 AI와 GPS"라는 새로운 능력만 없는 것입니다. 시리즈로 책을 쓴 보람이 여기서 드러납니다.

1.5 AI와 함께하는 시스템 아키텍처 설계

Vol.4는 단순히 "AI로 코드 짜는 책"이 아닙니다. 시스템 설계의 가장 처음 단계 — 부품 선정과 아키텍처 결정 — 부터 AI와 함께 토의합니다. 이 절에서 그 시작을 보입니다.

아래는 1장의 시스템 구성을 결정할 때 실제로 활용할 수 있는 프롬프트의 예입니다.

AI와 함께 설계하기

나는 RC 카에 카메라를 달아 객체를 인식하고, GPS로 위치를 기록한 뒤, MCU(Infineon TC237)로 모터를 제어하려고 한다. 라즈베리파이를 카메라/AI 처리용, TC237을 모터 제어용으로 나누는 아키텍처가 적절한가? 양산차 ADAS 구조와 비교해 장단점을 설명하고, 두 보드 사이 인터페이스로 UART/SPI/I2C 중 무엇이 맞는지 추천해 줘. 마지막으로 이 아키텍처의 잠재적인 약점도 3가지 짚어 줘.

AI는 이 질문에 대해 잘 정리된 답을 내놓을 것입니다. 그러나 우리가 검증해야 할 부분도 있습니다. 임베디드 개발자의 일은 "AI의 답을 받는 것"에서 끝나지 않고 "AI의 답이 우리 환경에서 옳은지 검증하는 것"까지입니다.

예를 들어 AI가 "UART가 가장 무난하다"고 추천했다면, 우리는 다음을 확인해야 합니다.

- UART 보레이트는 명령 송수신 빈도를 감당하기에 충분한가? (9600bps면 초당 약 100건의 명령. RC카 제어에는 충분.)
- 라즈베리파이의 UART 핀은 다른 용도(블루투스, GPS 등)와 충돌하지 않는가?
- 두 보드 사이 GND 공통화는 되어 있는가? (전원 분리시 자주 누락되는 실수.)

AI는 일반론을 압니다. 그러나 "내 환경의 구체적 제약"은 모릅니다. AI의 답을 받고 — 그 답을 내 시스템에 적용 가능한지 검증하는 것 — 이것이 "AI와 함께 설계한다"는 말의 진짜 의미입니다.

AI 와 함께 설계하기

위에서 라즈베리파이와 TC237 사이를 UART 로 연결한다고 했을 때, 라즈베리파이의 어떤 핀을 쓰는 것이 안전한가? 다음 제약 조건을 모두 고려해 줘: 1) GPS NEO-M8N 도 UART 로 라즈베리파이에 연결되어 있다 (/dev/serial0 점유) 2) 블루투스를 비활성화하지 않고 싶다 3) USB 슬롯은 카메라 USB 디버그용으로 1 개를 비워야 한다 구체적인 핀 번호와 함께, GPIO UART 를 추가로 활성화하는 방법 또는 USB-UART 어댑터를 쓰는 방법 중 무엇이 더 나은지 비교해 줘.

이런 식의 "제약 조건을 같이 알려주고 토의하는" 방식은 Vol.4 내내 반복됩니다. 처음에는 어색하지만, 몇 번 해보면 — AI 없이 혼자 설계하던 시절보다 결정의 속도와 품질이 모두 올라가는 것을 느끼게 됩니다.

Chapter 1 핵심 정리

MCU 는 강력하지만, 카메라·AI·GPS 영역은 SoC 의 영토다 — 연산량·메모리·OS 의 세 벽 때문.

실제 자동차 ADAS 도 카메라 ECU(SoC)와 차량 제어 ECU(MCU)를 분리한다 — 검증된 패턴.

Vol.4 RC 카는 라즈베리파이(인지) + TC237(제어) 구조로 간다.

Vol.3 의 TC237 펌웨어는 거의 그대로 재사용된다. 명령 출처만 PC → 라즈베리파이로 바뀐다.

AI 는 아키텍처 설계 단계부터 함께 토의하는 동료다. 단, 결정과 검증의 책임은 엔지니어에게 있다.

"제약 조건을 같이 알려주고 토의하는" 프롬프트 패턴이 Vol.4 내내 반복된다.

Chapter 2. 라즈베리파이 환경 구축 — 임베디드 개발자를 위한 리눅스 최소 지식

2.1 라즈베리파이 모델 선정 (Pi 4 vs Pi 5)

Vol.4 를 시작하면서 첫 번째 결정은 "어떤 라즈베리파이를 살 것인가"입니다. 단순한 질문 같지만, 발열·전력·예산·성능이 얹혀 있어 임베디드 시스템 설계의 첫 단추가 됩니다. 자동차 개발로 치면 "어떤 ECU 플랫폼을 쓸 것인가"에 해당하는 결정입니다.

두 모델의 핵심 차이를 표로 정리하면 다음과 같습니다.

Raspberry Pi 4 Model B vs Raspberry Pi 5 주요 사양 비교

항목	Raspberry Pi 4 Model B	Raspberry Pi 5	Vol.4 추천
 CPU	Cortex-A72 4코어 1.8GHz	Cortex-A76 4코어 2.4GHz	Pi5 성능 약 2~3배 향상
 RAM 옵션	2GB / 4GB / 8GB LPDDR4-3200	4GB / 8GB LPDDR4X-4267	8GB 권장 여유로운 멀티태스킹
 AI 추론 성능 (YOLOv8n 기준)	기준 성능 (예: 3~5 FPS @ 640x480)	약 2~3배 향상 (예: 7~15 FPS @ 640x480)	Pi5 실시간성 확보에 유리
 소비 전력 (일반 / 고부하)	약 3 ~ 8W (유휴 ~ 고부하)	약 4 ~ 12W+ (유휴 ~ 고부하, 상황에 따라)	Pi4 유리 전력 민감한 프로젝트
 발열	히트싱크 권장	팬 권장 (장시간 고부하시)	Pi5 주의 쿨링 설계 필수
 PCIe	없음	PCIe 2.0 x1 지원 (M.2 HAT+ 등 확장 가능)	Pi5 고속 확장에 유리
 카메라 / 디스플레이 커넥터	CSI/DSI 커넥터 2개 (2-lane MIPI)	CSI/DSI 커넥터 2개 (4-lane MIPI 지원)	둘 다 가능 카메라/디스플레이 호환
 USB	USB 3.0 x 2 USB 2.0 x 2	USB 3.0 x 2 USB 2.0 x 2	동일
 GPIO	40핀 (완벽 호환)	40핀 (완벽 호환)	동일
 가격대 (국내 기준)	약 6 ~ 10만원	약 10 ~ 14만원	예산에 따라 선택 성능/확장성은 Pi5 우위

★ 한 줄 요약

RC카의 저속 추적과 YOLOv8n 실행까지는 Pi4도 충분하지만, 실시간성이 중요하거나 향후 모델 확장을 고려한다면 **Pi5**가 더 여유 있는 선택입니다.



Vol.4 의 작업 부하 — 640x480 영상에서 YOLOv8n 모델 추론과 GPS 파싱과 UART 통신을 동시에 — 을 기준으로 보면, Pi 5 는 약 8~12 FPS, Pi 4 는 약 3~5 FPS 를 냅니다. RC 카가 천천히 움직이는 상황(시속 1~2 km)에서는 Pi 4 도 충분합니다. 빠른 추적이 필요하거나, 영상 해상도를 더 높이고 싶거나, 모델을 키우고 싶다면 Pi 5 가 낫습니다.

배터리와 발열 — 모바일 운용의 제약

RC 카는 모바일 시스템입니다. 콘센트에 꽂아 쓸 수 없습니다. Pi 5 는 최대 12W 를 소비하는데, 이는 5V/2.4A 정도입니다. 7.4V 1500mAh 리튬 배터리로 운영한다고 가정하면 — 라즈베리파이 + 카메라 + GPS + TC237 + 모터까지 모두 한 배터리에서 끌어 쓰면 30 분도 못 갑니다.

그래서 보통 두 가지 전략 중 하나를 씁니다. ① 라즈베리파이용 배터리와 TC237/모터용 배터리를 분리. ② 모든 것을 한 배터리에서 끌되, 라즈베리파이 전압을 일정하게 유지하기 위해 DC-DC 컨버터(예: LM2596 또는 더 효율 좋은 부품)를 통해 5V/3A 를 안정적으로 공급.

Vol.3 에서 우리가 "TC237 보드용 보조 배터리"를 분리한 결정을 떠올려 보세요 — 같은 원칙입니다. 노이즈와 전류 변동이 큰 모터 쪽과, 정밀한 제어 회로를 분리하는 것이 임베디드 모바일 시스템의 기본입니다. 라즈베리파이는 더더욱 분리하는 것이 안전합니다.

발열 관리:

Pi 5 는 케이스 내부에서 무팬으로 YOLO 추론을 5 분 이상 돌리면 thermal throttling 이 시작됩니다 (CPU 클럭이 자동 하강해 발열을 막는 메커니즘). RC 카에 올릴 때는 액티브 쿨링(작은 팬)이나 적어도 큰 히트싱크를 다는 것을 권장합니다.

2.2 OS 설치와 SSH·VNC 원격 접속

라즈베리파이를 RC 카에 올리면 모니터·키보드를 매번 연결할 수 없습니다. 처음부터 원격 접속을 전제로 셋업해야 합니다. 이것을 "헤드리스(headless) 셋업"이라고 부릅니다.

Raspberry Pi Imager 로 OS 굽기

공식 도구인 Raspberry Pi Imager(rpi.org 에서 무료 다운로드)를 사용해 microSD 카드(최소 32GB, Class 10 이상 권장)에 Raspberry Pi OS(64-bit, Bookworm)를 굽습니다. 굽기 직전 톱니바퀴 아이콘 또는 "Edit Settings" 화면에서 다음을 미리 설정하면 첫 부팅부터 헤드리스 사용이 가능합니다.

- 호스트네임 설정 (예: rccar.local)
- 사용자명과 비밀번호 설정 (기본 pi/raspberry 는 보안상 변경 권장)

- Wi-Fi SSID 와 비밀번호 설정
- Locale 설정 — Time Zone Asia/Seoul, Keyboard Layout US
- SSH 활성화 — "Use password authentication" 체크

이 설정 없이 OS 만 굽고 SD 카드를 꽂아 부팅하면 — 처음에는 모니터/키보드를 연결해야 SSH 가 켜집니다. 미리 설정하는 게 훨씬 편합니다.

첫 부팅과 SSH 접속

SD 카드를 라즈베리파이에 꽂고 전원을 인가하면, 몇 분 후 라우터의 클라이언트 목록에 새 기기가 나타납니다. PC 또는 노트북에서 다음 명령으로 접속합니다.

```
# macOS, Linux, Windows 10/11 PowerShell 에서
ssh pi@rccar.local

# .local 이 안 되면 IP 로 직접:
ssh pi@192.168.0.42
```

처음 접속 시 호스트 키 확인 질문에 yes 로 답합니다. 그 다음부터는 비밀번호만으로 접속됩니다. 매번 비밀번호 치기 귀찮다면 ssh-keygen 으로 키 페어를 만들고 ssh-copy-id pi@rccar.local 로 등록하세요. 이후 비밀번호 입력 없이 접속됩니다.

VNC 로 GUI 원격 접속

SSH 는 텍스트 콘솔만 보여줍니다. 카메라 영상을 미리보기로 확인하거나, GUI 도구 (예: 데이터 시각화)를 사용할 때는 VNC 가 편합니다.

```
# 라즈베리파이에서 VNC 활성화
sudo raspi-config
# Interfacing Options → VNC → Enable

# PC 에서 RealVNC Viewer 를 설치하고
# 호스트: rccar.local (또는 IP)
# 사용자/비밀번호로 접속
```

2.3 임베디드 개발자가 꼭 알아야 할 리눅스 명령어

MCU 개발만 해왔다면 리눅스 셸이 낯설 수 있습니다. Vol.4 의 코드를 따라가는 데

필요한 최소한의 명령어를 정리합니다. 외울 필요는 없습니다 — 자주 쓰다 보면 자연스럽게 손에 익습니다.

== 디렉토리·파일 ==

ls -la 현재 디렉토리의 모든 파일 (숨김 포함, 권한 표시)
cd /home/pi 디렉토리 이동 (cd ~ 는 홈 디렉토리)
pwd 현재 디렉토리 경로
cat file.txt 파일 내용 출력
nano file.txt 간단한 텍스트 편집기 (Ctrl+O 저장, Ctrl+X 종료)
cp src dst 파일 복사
mv src dst 파일 이동 또는 이름 변경
rm file 파일 삭제 (rm -rf 폴더 는 폴더 통째 삭제 - 주의!)
mkdir name 폴더 생성

== 권한·실행 ==

sudo cmd 관리자 권한으로 실행
chmod +x file.sh 실행 권한 부여
./file.sh 현재 디렉토리의 스크립트 실행

== 패키지 ==

sudo apt update 패키지 목록 갱신
sudo apt upgrade -y 모든 패키지 업그레이드
sudo apt install pkg-name 패키지 설치
sudo apt remove pkg-name 패키지 제거

== 서비스·로그 ==

systemctl status name 서비스 상태 확인
sudo systemctl start name 서비스 시작
sudo systemctl enable name 부팅 시 자동 시작 설정
journalctl -u name -f 서비스 로그 실시간 확인

== 모니터링 ==

top / htop CPU/메모리 사용량 모니터 (htop 은 별도 설치)
df -h 디스크 사용량
free -h 메모리 사용량
vcgencmd measure_temp CPU 온도 (라즈베리파이 전용)

== 네트워크 ==

ip a 네트워크 인터페이스 확인

ping rccar.local 호스트 도달성 확인

ss -tlnp 열린 TCP 포트 목록

임베디드 개발자가 자주 쓰는 패턴:

Vol.4 작업 중 가장 많이 치게 될 명령은 — vcgencmd measure_temp(CPU 온도 확인, 발열 추적), journalctl -u rccar.service -f (우리가 만든 서비스 로그 보기), 그리고 htop(어떤 프로세스가 CPU 를 잡아먹는지) 입니다.

2.4 Python 가상환경과 패키지 관리

Vol.4 의 모든 코드는 Python 으로 작성됩니다. 시스템 Python(/usr/bin/python3) 에 직접 패키지를 설치하면 OS 가 망가지기 쉬우므로 — 정확히 말하면 OS 의 다른 도구들과 버전 충돌이 일어나므로 — 반드시 가상환경(virtual environment)을 사용합니다.

가상환경 만들기

가상환경 생성 (한 번만)

```
python3 -m venv ~/rccar-env
```

활성화 (셸을 새로 열 때마다)

```
source ~/rccar-env/bin/activate
```

셸 프롬프트 앞에 (rccar-env)가 붙으면 성공

이 상태에서 pip install ... 하면 가상환경 안에만 설치됨

비활성화

```
deactivate
```

Vol.4 에 필요한 패키지 설치

가상환경 활성화한 상태에서

```
pip install --upgrade pip
```

```
pip install opencv-python==4.8.1.78
pip install ultralytics==8.3.0
pip install picamera2
pip install pyserial pynmea2
pip install numpy

# 설치 검증
python -c "import cv2, ultralytics, picamera2, serial, pynmea2; print('OK')"
```

패키지 버전을 명시한 이유는 — 시간이 지나면 라이브러리 API가 바뀌면서 책의 코드가 안 돌아갈 수 있기 때문입니다. 위 버전은 본 책 집필 시점 기준으로 검증된 조합입니다. 더 최신 버전을 쓰고 싶다면 — 코드도 그에 맞게 손봐야 할 수 있다는 점을 염두에 두세요.

requirements.txt — 환경을 기록하는 습관

패키지를 다 설치한 뒤에는 requirements.txt를 만들어 저장해 두는 습관을 들이세요. 나중에 SD 카드가 망가지거나 다른 라즈베리파이를 셋업할 때, 한 줄로 같은 환경을 재현할 수 있습니다.

```
# 현재 가상환경의 모든 패키지를 파일로 저장
pip freeze > ~/rccar/requirements.txt

# 새 환경에서 복원
python3 -m venv ~/rccar-env
source ~/rccar-env/bin/activate
pip install -r ~/rccar/requirements.txt
```

2.5 systemd로 부팅 시 자동 실행 설정하기

RC카는 보통 "전원만 켜면 알아서 동작하는" 시스템을 원합니다. SSH로 접속해 매번 python main.py를 칠 수 없습니다. systemd를 이용해 부팅 시 자동 실행되는 서비스로 등록하는 법을 미리 알아 둡니다.

```
# /etc/systemd/system/rccar.service 파일 작성 (sudo nano로)
```

```
[Unit]
Description=RC Car AI Tracker
After=network.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/rccar
ExecStart=/home/pi/rccar-env/bin/python /home/pi/rccar/main.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
# 등록 및 시작
sudo systemctl daemon-reload
sudo systemctl enable rccar.service # 부팅 시 자동 시작
sudo systemctl start rccar.service # 지금 바로 시작

# 상태 확인
systemctl status rccar.service

# 실시간 로그 보기
journalctl -u rccar.service -f
```

이 서비스가 등록되어 있으면 — 코드가 크래시해도 5 초 후 자동 재시작되고, 라즈베리파이를 껐다 켜도 자동 시작됩니다. 양산 임베디드 시스템에서 watchdog 과 비슷한 역할을 합니다.

2.6 AI 와 함께하는 라즈베리파이 초기 셋업

초기 셋업을 매번 손으로 하면 시간이 들고 빠뜨리기도 쉽습니다. AI 에게 셋업 스크립트 초안을 만들어달라고 부탁하는 것이 좋은 시작입니다.



AI 와 함께 설계하기

라즈베리파이 4 Model B 에 Raspberry Pi OS(64-bit, Bookworm)가 막 설치된 상태다. 다음을 자동화하는 bash 스크립트를 작성해 줘: 1) 시스템 업데이트 (apt

update + upgrade) 2) http, vim, git, build-essential 설치 3) Python 가상환경 ~/rccar-env 생성 4) opencv-python, ultralytics, picamera2, pyserial, pynmea2 를 가상환경에 설치 5) 카메라 인터페이스 활성화 확인 (raspi-config nonint) 6) UART 인터페이스 활성화 + Bluetooth 비활성화 (GPS 용) 7) 설치 검증 — 각 라이브러리 import 가능 여부 출력 8) 마지막으로 sudo reboot 권장 메시지 출력 스크립트는 먹등성 (여러 번 실행해도 안전)을 가져야 한다.

AI 가 만든 스크립트는 그대로 실행하기 전에 한 줄씩 읽어보세요. 특히 sudo 가 붙은 명령은 시스템 설정을 바꾸므로, 의도와 부작용을 확인한 뒤 실행하는 것이 임베디드 개발자의 습관이어야 합니다. AI 가 "이 명령은 X 를 한다"고 주석으로 설명해 두지 않았다면, AI 에게 다시 물어 한 줄씩 설명을 받아내세요.

AI 와 함께 설계하기

위 스크립트에서 'dtoverlay=disable-bt' 라인을 /boot/firmware/config.txt 에 추가한다고 했다. 이 한 줄이 정확히 무엇을 하는지, 만약 잘못 적용되면 어떤 문제가 생길 수 있는지, 그리고 잘못 적용했을 때 복구하는 방법까지 설명해 줘.

"한 줄씩 설명을 요청하고 검증하는 습관" — 이것이 임베디드 개발자가 AI 를 안전하게 활용하는 핵심입니다. AI 는 빠르지만 가끔 틀리고, 우리 시스템의 모든 제약은 모릅니다. 우리가 검증해야 합니다.

Chapter 2 핵심 정리

Vol.4 작업 부하에는 Pi 4 도 가능하지만, Pi 5 가 추론 FPS 와 발열 모두에서 유리하다. 처음부터 헤드리스 셋업을 전제로 — Imager 의 Edit Settings 에서 SSH·Wi-Fi 미리 설정.

임베디드 개발자도 ls/cd/cat/nano/sudo/apt/systemctl 정도는 익숙해져야 한다.

Python 패키지는 반드시 가상환경(venv)에 설치한다.

requirements.txt 로 환경을 기록해 두면 SD 카드를 새로 구워도 재현 가능하다.

RC 카는 systemd 서비스로 등록해 부팅 시 자동 실행되게 만든다.

셋업 스크립트는 AI 로 초안을 만들고, 한 줄씩 검증해서 실행한다.

Chapter 3. 카메라 — 라즈베리파이의 눈을 뜨다 (파란공 도전)

이 챕터에서 우리는 라즈베리파이에 "눈"을 달아 줍니다. 그 첫 번째 도전은 — 신경망 같은 무거운 도구 없이도 — 단순한 색깔만으로 물체를 찾아 추적해 보는 것입니다. 파란공이 우리의 첫 추적 대상입니다. 단순한 시작이지만, 여기서 배우는 "바운딩 박스와 좌표" 개념은 4 장의 YOLO 에서도, 8 장의 통합 시스템에서도 똑같이 쓰입니다. 가장 작은 단위로 시작해 점점 올라가는 — 그것이 Vol.4 의 학습 방식입니다.

3.1 Pi Camera vs USB 웹캠 — 선택 기준

라즈베리파이에 카메라를 다는 방법은 크게 두 가지입니다.

Pi Camera (CSI 인터페이스)

라즈베리파이 보드의 전용 MIPI CSI 커넥터에 연결합니다. MIPI 는 휴대폰에서 모듈 카메라가 SoC 와 통신할 때 쓰는 고속 시리얼 규격입니다. 데이터 경로가 짧고, 라즈베리파이의 ISP(Image Signal Processor)와 직접 연동되어 효율이 좋습니다. 화이트 밸런스·노출 같은 처리도 하드웨어가 해 줍니다.

종류:

- Camera Module V2 (Sony IMX219, 8MP) — 가장 흔하고 저렴, 약 4 만원.
- HQ Camera (Sony IMX477, 12MP) — 교환식 렌즈 마운트(C/CS) 지원, 8 만원.
- Camera Module 3 (Sony IMX708, 12MP) — 오토포커스 + HDR, 6 만원.

RC 카에는 Camera Module V2 또는 V3 가 적합합니다. 크기가 작고(약 25x24mm) 가벼우며, 차체에 장착하기 용이합니다.

USB 웹캠 (UVC 클래스)

Logitech C270, C920 같은 일반 USB 웹캠도 라즈베리파이에서 plug-and-play 로 인식됩니다. 설치가 간단하고 가격 선택폭이 넓다는 장점이 있지만, 단점도 있습니다.

- USB 대역폭과 추가 지연 — Pi Camera 대비 30~80ms 더 느림.
- USB 포트 점유 — 다른 USB 장치(USB-UART 어댑터 등)와 자리 경쟁.

- 크기와 케이스 — RC 카에 장착하기에는 부피가 크고 마운트가 불편.

Vol.4 RC 카에는 Pi Camera 계열을 권장합니다. 다만 "이미 갖고 있는 USB 웹캠이 있다" 면 그것으로 시작해도 충분합니다 — 추적 성능에 미세한 차이일 뿐, 책의 모든 코드는 양쪽 모두에서 동작합니다.

3.2 libcamera/picamera2 로 영상 캡처하기

라즈베리파이의 카메라 스택은 2022 년경 큰 변화를 겪었습니다. 그 이전까지는 picamera 라는 라이브러리가 표준이었지만, Raspberry Pi OS Bookworm(2023~)부터는 libcamera 기반의 picamera2 가 표준이 되었습니다. 옛 picamera 는 더 이상 동작하지 않으므로 — 인터넷에서 picamera 예제를 발견했다면 의심하고 picamera2 버전을 찾아야 합니다.

기본 캡처 코드

```
from picamera2 import Picamera2
import cv2

picam2 = Picamera2()
config = picam2.create_preview_configuration(
    main={"size": (640, 480), "format": "RGB888"}
)
picam2.configure(config)
picam2.start()

while True:
    frame = picam2.capture_array()
    # picamera2 는 기본적으로 RGB 로 주는데, OpenCV 는 BGR 을 쓰므로 변환
    frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
    cv2.imshow("Pi Camera", frame_bgr)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

picam2.stop()
cv2.destroyAllWindows()
```

RGB 와 BGR 의 순서 — 혼한 함정:

picamera2 는 RGB 순서로 픽셀을 줍니다. OpenCV 의 거의 모든 함수는 BGR 순서를 가정합니다. 이 두 가지를 섞으면 빨간 물체가 파랗게 보이거나 그 반대가 됩니다. 영상이 이상하게 보이면 항상 첫 번째로 의심해야 할 것 — RGB/BGR 순서.

프레임 레이트 조정

기본 설정은 보통 30 FPS 정도입니다. 그러나 YOLO 추론이 5 FPS 밖에 안 나오는 상황에서 카메라가 30 FPS 로 들어오면 — 매 프레임을 버려야 합니다. 카메라 측에서 미리 FPS 를 낮추면 전력과 처리 부하를 모두 줄일 수 있습니다.

```
config = picam2.create_preview_configuration(
    main={"size": (640, 480), "format": "RGB888"},
    controls={"FrameRate": 10}, # 초당 10 프레임으로 제한
)
```

3.3 OpenCV 기초 — 영상은 어떻게 숫자로 표현되는가?

4 장에서 신경망이 어떻게 고양이를 보는지 다루기 전에, 영상이 컴퓨터에게 어떤 모습인지 짚고 갑니다. 이 절은 매우 짧지만 중요합니다 — 4 장의 신경망 설명이 "숫자 더미를 받아서 숫자 더미를 내놓는 함수"라는 관점을 가지려면 — 영상이 이미 숫자 더미라는 것을 자연스럽게 받아들이야 합니다.

컬러 영상은 본질적으로 3 차원 배열입니다. (높이, 너비, 채널) 크기의 직사각형 입체입니다.

```
import cv2
import numpy as np

img = cv2.imread("ball.jpg")
print(img.shape)    # 예: (480, 640, 3)
print(img.dtype)    # uint8 — 0~255 정수
print(img[100, 200]) # 위치(100,200)의 BGR 값, 예: [180, 30, 25] (파란공)
print(img[100, 200, 0]) # B 채널만, 180
print(img[100, 200, 1]) # G 채널만, 30
print(img[100, 200, 2]) # R 채널만, 25
```

이 한 줄 — `img[100, 200]`이 `[180, 30, 25]` — 이 모든 컴퓨터 비전의 출발점입니다. 카메라가 보는 모든 것은 이렇게 작은 숫자 덩어리입니다. 차선 인식도, 고양이 검출도, ChatGPT 같은 멀티모달 모델도 — 모두 이 숫자 덩어리를 입력으로 받아 답을 내놓는 함수일 뿐입니다.

640x480 컬러 영상 한 장의 픽셀 수는 $= 640 \times 480 \times 3 = 921,600$ 개입니다. 30 FPS 로 들어오는 영상이라면 초당 약 2,800 만 개의 숫자가 우리 코드를 통과합니다. 이 "숫자의 강물"을 처리하는 효율이 — 곧 카메라 시스템의 성능입니다.

3.4 HSV 색공간으로 파란색 물체 검출하기

BGR 로는 "파란색"이라는 개념을 표현하기 어렵습니다. 같은 파란공이라도 햇빛 아래에서는 `[200, 50, 30]` 같은 값이 나오고, 그늘에서는 `[80, 20, 10]` 정도가 나옵니다. 단순히 "B 채널이 100 이상이고 R 채널이 50 이하"라는 규칙으로는 그들의 파란공을 놓칩니다.

HSV 의 발상

HSV(Hue, Saturation, Value)는 "색조"를 분리해서 표현하는 색공간입니다.

- Hue(색조) — 무지개색 중 어느 위치인가. 빨강이 0, 노랑이 60, 초록이 120, 파랑이 240, 보라가 300 (0~360 도).
- Saturation(채도) — 얼마나 진한가. 0 이면 흑백, 255 면 가장 선명.
- Value(명도) — 얼마나 밝은가. 0 이면 검정, 255 면 가장 밝음.

같은 파란공이라면 조명이 바뀌어도 Hue 는 거의 일정하게 유지됩니다. Saturation 과 Value 만 변합니다. 그래서 "색깔로 물체 찾기"는 HSV 에서 훨씬 견고합니다. 휴대폰의 "빨간 눈 보정" 같은 기능도 같은 원리로 작동합니다.

OpenCV 의 Hue 범위 — 또 다른 함정:

OpenCV 는 Hue 를 0~360 이 아니라 0~180 으로 표현합니다 (한 바이트에 담기 위해 절반으로 나눔). 그래서 파란색의 Hue 임계값은 240 의 절반인 약 110~130 입니다. 인터넷의 HSV 색상표를 0~360 으로 보고 그대로 쓰면 안 됩니다.

실제 코드

```
import cv2
```

```

import numpy as np

img = cv2.imread("ball.jpg")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# OpenCV의 Hue는 0~179
# 파란색의 Hue는 대략 100~130
lower_blue = np.array([100, 100, 50])
upper_blue = np.array([130, 255, 255])

mask = cv2.inRange(hsv, lower_blue, upper_blue)
# mask는 흑백 이미지: 파란색 픽셀은 255, 나머지는 0

result = cv2.bitwise_and(img, img, mask=mask)
# 원본에서 파란색만 남기고 나머지를 검정으로

cv2.imwrite("mask.jpg", mask)
cv2.imwrite("result.jpg", result)

```

Morphology — 잡음 정리

실제로 코드를 돌려보면 마스크에 작은 "점" 노이즈가 박혀 있는 것을 보게 됩니다. 비슷한 색조의 다른 물체(예: 파란 카펫의 일부)가 잡혀 들어오기 때문입니다. 이 작은 노이즈를 제거하는 표준 기법이 Morphology입니다.

```

kernel = np.ones((5, 5), np.uint8)
mask_clean = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
# Opening = Erode(작은 영역 깎기) + Dilate(다시 부풀리기)
# → 작은 점은 사라지고 큰 영역은 유지

```

3.5 바운딩 박스와 중심 좌표 구하기

마스크에서 파란색 영역을 찾았다면, 다음은 그 영역이 화면의 어디에 있는지를 알아내는 일입니다. 이 "위치 정보"가 4장에서 YOLO를 만났을 때도, 8장에서 RC 카를 조향할 때도 똑같이 쓰이는 핵심입니다.

```

contours, _ = cv2.findContours(mask_clean, cv2.RETR_EXTERNAL,

```

```

cv2.CHAIN_APPROX_SIMPLE)

if contours:
    # 가장 큰 컨투어(가장 큰 파란 영역)를 선택
    largest = max(contours, key=cv2.contourArea)

    # 면적이 너무 작으면 무시 (노이즈일 가능성)
    if cv2.contourArea(largest) > 500:
        x, y, w, h = cv2.boundingRect(largest)
        cx = x + w // 2 # 중심 X
        cy = y + h // 2 # 중심 Y
        print(f"파란공 발견: 중심=({cx}, {cy}), 크기={w}x{h}")

    # 시각화: 원본에 박스와 중심점 그리기
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.circle(img, (cx, cy), 5, (0, 0, 255), -1)

```

이 다섯 개의 숫자 — x , y , w , h , 그리고 (cx, cy) — 가 컴퓨터 비전의 표준 출력입니다. YOLO 도 정확히 같은 형태로 결과를 줍니다. 클래스 라벨이 "blue_ball"에서 "cat"으로 바뀔 뿐, 구조는 동일합니다.

Vol.3 → Vol.4 연결고리:

Vol.3 에서 IMU 로 '내가 어디를 향하고 있는가'를 알았다면, Vol.4 에서는 카메라로 '내 앞의 무엇이 어디 있는가'를 압니다. 자세 추정(Vol.3) + 객체 위치(Vol.4) = 자율 추적. 양산차의 자율주행이 정확히 이 두 가지를 합치는 일입니다.

3.6 화면 좌표를 의미 있는 정보로 변환하기

(cx, cy) 는 단순한 좌표입니다. 8 장에서 이것을 RC 카 조향 명령으로 변환할 것이지만, 그 전에 한 가지 사고 실험을 해봅시다 — "이 좌표가 화면의 어느 쪽인가"를 정량화하는 일입니다.

```

W = 640 # 화면 가로 폭
H = 480 # 화면 세로 폭

# 좌우 위치 정량화 (-1.0 ~ +1.0)

```

```

norm_x = (cx - W/2) / (W/2)
# 음수면 왼쪽, 양수면 오른쪽, 0 이면 정중앙

# 텍스트 라벨
if norm_x < -0.2:
    direction = "LEFT"
elif norm_x > 0.2:
    direction = "RIGHT"
else:
    direction = "CENTER"

# 크기 = 거리의 단서
size_ratio = w / W
# 0.5 이상이면 화면의 절반 이상 — 매우 가까움

```

이렇게 "화면 좌표 → 의미 있는 변수"로 추상화하면, 8장에서 RC 카 제어 로직을 짤 때 — 단순히 "norm_x 값에 비례해 조향" 같은 직관적인 코드를 쓸 수 있습니다. 이런 추상화 계층을 두는 것이 임베디드 시스템 설계의 정석입니다.

3.7 AI와 함께하는 파란공 검출 실습

이번 챕터의 마지막 실습입니다. AI와 함께 "파란공을 따라가는 시각화 프로그램"을 만들어 봅시다. 카메라 영상에 바운딩 박스를 그리고, 공이 화면 좌우 어느 쪽에 있는지를 출력합니다.

AI와 함께 설계하기

라즈베리파이 + Pi Camera (Bookworm OS, picamera2) 환경에서 다음을 하는 Python 코드를 작성해 줘: 1) 영상 해상도 640x480, 10 FPS 2) HSV 색공간에서 파란색 검출 (Hue 100~130) 3) Morphology Opening 으로 노이즈 제거 4) 가장 큰 파란색 영역의 바운딩 박스를 그리고 중심에 점 표시 5) 화면 가운데 세로선을 그리고, 공의 중심 위치를 LEFT/CENTER/RIGHT 로 화면에 표시 6) FPS 를 좌측 상단에 표시 7) 'q' 키로 종료, 종료 시 카메라 자원 안전하게 해제 전체 코드를 보여주고, HSV 임계값 튜닝 팁도 같이 알려줘.

AI가 만든 코드를 실행했을 때 검출이 잘 안 된다면, 다음 박스의 디버깅 프롬프트를 활용해 보세요.

AI 와 함께 설계하기

위 코드를 실행했더니 햇빛 아래에서는 파란공을 잘 찾는데, 형광등 아래 실내에서는 거의 찾지 못한다. 다음을 알려줘: 1) HSV 임계값을 어떻게 조정해야 할까? Hue, Saturation, Value 각각의 적절한 범위는? 2) 조명에 더 견고한 검출을 위한 추가 기법 3 가지 (예: morphology, 컬러 캘리브레이션, 적응형 임계값 등) 3) 임계값을 자동으로 찾아주는 GUI 슬라이더 도구를 만드는 짧은 코드

"AI 에게 추가 도구를 만들어 달라고 요청하는 것" — 이런 메타 기법이 Vol.4 에서 자주 등장합니다. AI 는 "답을 주는 존재"가 아니라 "내 작업을 가속하는 도구를 만들어 주는 존재" 로도 쓸 수 있습니다.

Chapter 3 핵심 정리

Pi Camera 는 CSI 로 직접 연결되어 USB 웹캠보다 지연이 적다 — RC 카에 적합.
Bookworm 이후로는 picamera2 가 표준이다 (picamera 는 사용하지 말 것).
picamera2 는 RGB, OpenCV 는 BGR — 채널 순서 주의.
영상은 본질적으로 (H, W, 3) 크기의 uint8 배열이다 — 이것이 컴퓨터 비전의 출발점.
BGR 보다 HSV 가 조명 변화에 견고하다 — 색깔 기반 검출에는 HSV 를 쓴다.
OpenCV 의 Hue 범위는 0~179 (인터넷 색상표의 절반).
바운딩 박스 (x, y, w, h)와 중심 좌표 (cx, cy)는 색상 검출이든 YOLO 든 똑같은 출력이다.
AI 에게 코드뿐 아니라 디버깅·튜닝 도구까지 같이 요청하면 개발 속도가 크게 빨라진다.

Chapter 4. YOLOv8 — 객체 인식의 표준을 만나다 (고양이 1 차 도전)

3 장에서 우리는 "색깔로 물체를 찾는" 가장 단순한 방법을 배웠습니다. 그러나 고양이는 흰색일 수도, 검은색일 수도, 얼룩무늬일 수도 있습니다. 단순한 색상 마스크로는 어렵습니다. 우리에게 필요한 것은 — "모양과 패턴을 이해하는" 기술입니다. 그것이 신경망이고, YOLO 입니다. 그러나 YOLO 를 도구로 쓰기 전에 한 가지를 짚고 가야 합니다 — 신경망이 도대체 어떻게 고양이를 "본다"는 것인가? 이 절을 잘 이해하고 넘어가면, Vol.4 나머지가 훨씬 명료해집니다.

4.1 객체 인식이란 무엇인가 — Classification vs Detection

"객체 인식"이라는 말은 흔히 두 가지 다른 작업을 한 단어로 묶어 부릅니다. 우리는 이 둘을 정확히 구분해야 합니다.

Classification (분류)

사진 한 장이 주어졌을 때 "이 사진의 주인공은 무엇인가?"에 답합니다. 출력은 하나의 라벨입니다 — 예: "cat", "dog", "car". 사진에서 어디에 있는지는 모릅니다. Classification 은 신경망의 첫 성공 사례였습니다. 2012 년 ImageNet 대회에서 AlexNet 이 압도적 성능으로 우승하면서 — 현재의 AI 붐이 시작된 출발점입니다.

Detection (검출)

사진에 어떤 물체들이 있고, 각각 어디에 있는지를 모두 찾습니다. 출력은 여러 개의 (라벨, 바운딩 박스, 신뢰도) 쌍입니다. 한 사진에 고양이 두 마리와 사람 한 명이 있다면 — Detection 은 세 개의 결과를 줍니다.

RC 카가 고양이를 따라가려면 "고양이가 있다"는 사실만으로는 부족합니다 — "고양이가 어디에 있다"가 필요합니다. 그래서 Detection 이 답입니다. 그 분야의 대표주자가 바로 YOLO 입니다.

참고로 더 나아간 작업:

Segmentation = 픽셀 단위로 "이 픽셀은 고양이의 일부"를 표시. Pose Estimation = 사람/동물의 관절 위치를 찾기. 이런 더 정교한 작업도 YOLO 시리즈에 모두 포함되어 있습니다. Vol.4 는 Detection 까지만 다룹니다.

4.2 픽셀에서 고양이까지 — 컴퓨터는 어떻게 고양이를 보는가

YOLO 를 도구로 쓰기 전에 짚고 갈 것이 있습니다. 신경망은 정확히 어떻게 고양이를 "보는" 걸까요? 이 절은 Vol.4 전체에서 가장 중요한 절일 수 있습니다 — AI 를 "마법 상자"로 두느냐, "이해 가능한 기계"로 두느냐가 여기서 갈립니다.

4.2.1 모든 것의 시작 — RGB 픽셀값

3 장에서 본 것처럼, 카메라가 찍은 고양이 사진은 컴퓨터에게는 단지 (H, W, 3) 크기의 숫자 배열입니다. 640x480 컬러 이미지라면 92 만 1,600 개의 숫자입니다. 이 숫자 더미가 신경망의 입력입니다.

신경망의 임무는 — 이 92 만 개의 숫자를 받아서, 마지막에 "이건 cat 이고, 좌표는 (x, y, w, h)다"라는 짧은 답을 내놓는 것입니다. 92 만 개에서 다섯 개로의 "압축"이라고 봐도 됩니다. 그 과정에서 핵심 정보(객체와 위치)는 살리고, 부수적 정보(배경 텍스처 등)는 버립니다.

이 일을 하는 신경망의 가중치 개수는 — 가장 작은 YOLOv8n 조차 약 320 만 개입니다. 큰 모델인 YOLOv8x 는 6,800 만 개입니다. 이 수많은 숫자가 — 학습 과정에서 자동으로 정해집니다. 사람이 일일이 정하지 않습니다. "학습"이 무엇인지는 4.2.7 에서 다룹니다.

4.2.2 첫 번째 깨달음 — "고양이는 특정 픽셀 패턴이다"

고양이 사진들에는 어떤 공통점이 있을까요? 사람은 "털, 귀, 눈, 코, 수염"을 떠올립니다. 하지만 신경망은 이 단어들을 모릅니다. 신경망이 발견할 수 있는 것은 — 픽셀 값들 사이의 패턴뿐입니다.

예를 들어 "고양이 귀"라는 부분은 어떤 픽셀 패턴일까요? 삼각형 모양의 윤곽, 안쪽은 분홍색이거나 어두운 색, 가장자리는 털 질감 — 이것을 픽셀로 표현하면, "이 위치에서 그 위치로 갈수록 색이 급격히 바뀌고, 그 변화가 삼각형 모양을 그린다"가 됩니다. 신경망이 발견하는 것은 정확히 이런 패턴입니다.

재미있는 점은 — 사람도 같은 방식으로 본다는 것입니다. 신경과학에서 시각피질의 1 차 영역(V1)은 "엣지의 방향"을 검출한다고 알려져 있습니다. 그 다음 영역(V2)은 "엣지의 조합"을, 더 높은 영역은 "얼굴, 손, 도구" 같은 복합 객체를 검출합니다. 신경망의 계층 구조가 — 우연이 아니라 — 인간 시각계의 구조에서 영감을 받은 것입니다.

4.2.3 뉴런 하나의 일 — 가중치 곱하고 더하기

신경망의 가장 작은 단위는 뉴런입니다. 뉴런이 하는 일은 놀랄 만큼 단순합니다.

뉴런의 출력 = (입력 1 × 가중치 1) + (입력 2 × 가중치 2) + ... + 편향

코드로 쓰면:

```
output = w1*x1 + w2*x2 + w3*x3 + ... + b
```

이게 전부입니다. 입력 숫자들에 "가중치"라는 자기만의 숫자를 곱하고, 다 더해서, 마지막에 편향을 더합니다.

그러면 가중치란 무엇인가요? 가중치는 그 뉴런이 "무엇에 주목하는가"를 결정하는 숫자입니다. 어떤 뉴런은 가중치가 "왼쪽에서 오른쪽으로 색이 어두워지는 패턴"에 큰 값이 되도록 학습됩니다. 그러면 그 뉴런은 "세로 엣지 검출기"가 됩니다. 다른 뉴런은 "분홍색 삼각형"에 반응하도록 학습됩니다. 그러면 그 뉴런은 "고양이 귀 검출기"에 가까워집니다.

임베디드 개발자의 직관:

이건 디지털 필터와 정확히 같은 구조입니다. FIR 필터의 출력 $y[n] = h[0]*x[n] + h[1]*x[n-1] + \dots$ 는 입력에 계수(가중치)를 곱해 더하는 형태입니다. 신경망 뉴런 = 가중치를 학습으로 정하는 디지털 필터. 이 비유가 정확하지는 않지만 — 임베디드 개발자의 머릿속에 자리 잡는 첫 그림으로는 매우 유용합니다.

4.2.4 비선형성의 마법 — ReLU 와 "이건 귀 같다"는 판단

뉴런의 합계가 그대로 출력되지는 않습니다. 마지막에 "활성화 함수"를 통과시킵니다. 가장 흔히 쓰는 것이 ReLU(Rectified Linear Unit)입니다.

$\text{ReLU}(x) = \max(0, x)$

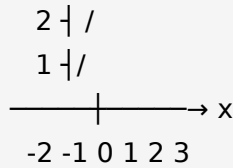
즉:

입력이 양수면 그대로 통과.

입력이 음수면 0 으로 끊어버린다.

그래프:

|
3 1 /



왜 이렇게 잔인하게 자르는 걸까요? 이유는 단순합니다 — "이건 귀 같다" / "아니다"라는 이진적 판단을 만들기 위해서입니다. 가중치 합이 양수라는 것은 "내가 찾던 패턴이 있다"는 신호이고, 음수라는 것은 "없다"는 신호입니다. ReLU는 "있는 신호만 다음 층으로 보낸다"는 뜻입니다.

이 비선형성이 없다면 — 아무리 많은 층을 쌓아도 결국 "입력의 가중치 합" 한 번에 불과합니다. 수학적으로 증명되는 사실입니다. 비선형성이 있어야 신경망은 "엣지 발견 → 귀 인식 → 고양이 판별"이라는 복합 추론을 할 수 있습니다.

ReLU 말고도 다른 활성화 함수들이 있습니다 — Sigmoid, Tanh, Leaky ReLU, GELU 등. 각각의 특성과 용도가 다르지만, Vol.4 수준에서는 "ReLU 비슷한 것을 쓴다" 정도로 충분합니다.

4.2.5 층을 쌓는 이유 — 픽셀 → 엣지 → 귀·눈 → 고양이

뉴런 하나로는 "고양이다"라고 판단할 수 없습니다. 그래서 뉴런들을 층(layer)으로 쌓고, 여러 층을 겹쳐 깊은 신경망(Deep Neural Network)을 만듭니다.

층을 쌓을 때 흥미로운 일이 일어납니다 — 층마다 "보는 것"의 추상도가 달라집니다.

Layer 1: 픽셀 강도 차이 → 엣지(edge)를 본다 (가로선, 세로선, 대각선)
 Layer 2: 엣지의 조합 → 모서리, 곡선, 텍스처를 본다
 Layer 3: 모서리 조합 → 눈, 귀, 코, 입 같은 부품을 본다
 Layer 4: 부품의 조합 → 얼굴, 몸통을 본다
 Layer 5: 몸통의 조합 → 고양이, 개, 자동차를 본다
 ⋮
 (YOLOv8n 은 약 200 층, YOLOv8x 는 약 350 층)

이건 누가 시켜서 그렇게 되는 것이 아닙니다. 학습 과정에서 "이렇게 역할 분담하는 게 가장 잘 맞춘다"는 것이 자동으로 결정됩니다. 신기하지요. 사람의 뇌도 비슷한 계층 구조로 시각 정보를 처리한다는 것이 신경과학에서 밝혀져 있습니다.

실제로 학습된 신경망의 중간 층을 "시각화"해 보면 — 1 층은 줄무늬 패턴, 2~3

층은 텍스처 패턴, 더 깊은 층은 "사람 얼굴 같은 무언가" 또는 "고양이 얼굴 같은 무언가"가 나타납니다. AI가 "무엇을 보고 있는지" 사람이 들여다볼 수 있는 — 매우 흥미로운 연구 분야입니다.

4.2.6 CNN — 사진의 어디든 같은 필터로 보기

그런데 이런 의문이 들 수 있습니다 — 고양이가 사진의 왼쪽에 있을 수도, 오른쪽에 있을 수도 있는데, 위치마다 다른 뉴런을 뒀야 하나요? 위치마다 "이 자리에 귀가 있나?"를 따로 학습시켜야 한다면, 학습할 가중치가 무한정 늘어납니다.

그건 너무 비효율적입니다. 그래서 컴퓨터 비전에서는 CNN(Convolutional Neural Network)을 씁니다. 핵심 아이디어는 — "같은 작은 필터를 사진 전체에 슬라이딩하면서 적용한다."

3x3 필터 한 개가 있다고 하자. 가중치가 다음과 같다면:

```
-1 -1 -1
 2  2  2
-1 -1 -1
```

이 필터는 "가로 엣지 검출기"가 된다.

(가운데 가로줄에서 큰 값, 위아래에서 작은 값일 때 합이 커진다)

사진의 왼쪽 위에 적용하든, 오른쪽 아래에 적용하든
가로 엣지가 있는 곳에서 큰 출력이 나온다.

원본 영상		필터 적용 결과
평평한 영역	→	0 (작음)
가로 엣지	→	큰 양수
평평한 영역	→	0 (작음)

이 방식이 "위치 불변성(translation invariance)"을 만듭니다. 고양이가 사진의 어디에 있든, 같은 필터가 찾아냅니다. 신경망 전체의 가중치 수도 극적으로 줄어듭니다 — 위치마다 다른 가중치를 가질 필요가 없으니까요.

CNN의 발견(LeCun, 1989) 이후 영상 인식 정확도는 극적으로 올라갔습니다. 그

이전까지는 사람이 손으로 "이런 패턴을 찾자"는 규칙(예: SIFT, HOG)을 짜 넣었지만 — CNN 은 "어떤 패턴을 찾을지"조차 학습으로 결정합니다. 영상 인식의 게임 규칙이 바뀐 순간입니다.

4.2.7 학습이란 무엇인가 — 가중치를 조금씩 바꾸기

그러면 가중치는 누가 정해줍니까? 사람이 일일이 정하지 않습니다. 학습(training)이라는 과정으로 자동으로 찾아잡니다.

학습의 큰 그림은 이렇습니다.

- ① 가중치를 랜덤한 작은 숫자로 초기화한다. (왜냐하면 모든 가중치가 0 이면 — 모든 뉴런이 똑같은 일을 하게 되어 학습이 안 됨)
- ② 학습 데이터 (사진, 정답) 한 쌍을 신경망에 입력한다.
- ③ 신경망이 답을 내놓는다. 처음에는 거의 다 틀린다.
- ④ "정답과 얼마나 다른가"를 손실(loss)이라는 숫자로 측정한다.
- ⑤ 손실을 줄이는 방향으로 모든 가중치를 "조금" 바꾼다.
- ⑥ ②~⑤ 를 수천만 번 반복한다.

⑤ 가 어떻게 "손실을 줄이는 방향"을 알아내는지 — 그것이 역전파(backpropagation)와 경사 하강법(gradient descent)입니다. 미분의 연쇄법칙을 이용해 각 가중치가 손실에 얼마나 기여하는지 계산하고, 그 반대 방향으로 가중치를 조금씩 옮깁니다.

이 책에서는 수식보다 직관을 우선합니다 — "손실 함수의 표면을 산이라 생각하면, 가장 가파른 내리막 방향으로 한 걸음씩 내려가는 것"이 학습입니다. 안개 속에서 산을 내려가는 사람을 떠올려 보세요 — 보이는 것은 발 밑의 기울기뿐이지만, 그 기울기를 따라 계속 내려가면 결국 골짜기(=낮은 손실)에 도달합니다.

실제 자동차 비유:

Mobileye 가 차선 인식기를 학습시킬 때도 같은 일을 합니다. 수백만 장의 도로 사진과 "이 픽셀은 차선이다"라는 정답을 짝지어 보여주고, 모델의 가중치를 조금씩 바꿔 가며 정답에 가까워지게 만듭니다. 우리가 6 장에서 할 파인튜닝이 정확히 이 일의 축소판입니다 — 다만 우리는 처음부터 학습하지 않고, 이미 학습된 모델의 가중치를 "미세 조정"하기만 합니다.

4.2.8 정리 — "AI 는 마법이 아니라 미분 가능한 함수"

이제 신경망이 더 이상 마법으로 보이지 않을 것입니다. 정리하면 — 신경망은 이런 함수입니다.

입력: 사진의 픽셀 숫자들 (예: 92 만 개)

↓ ($W1 \times \text{입력} + b1$, 그 다음 ReLU, 그리고 CNN 슬라이딩)

중간: 추상적 특징들 (예: 엷지, 모서리)

↓ ($W2 \times \text{중간} + b2$, ReLU)

중간: 더 추상적 특징들 (예: 귀, 눈)

↓ ... (수십~수백 층) ...

출력: "cat 이고 좌표는 (x,y,w,h)"

학습 = 이 W 들과 b 들을 정답에 맞도록 조정하는 일.

YOLOv8 도 본질적으로 같습니다. 더 정교한 구조와 약 320 만~6800 만 개의 가중치를 가지고 있지만, 픽셀이 들어가서 라벨과 좌표가 나오는 함수라는 점은 같습니다. 이제 우리는 그 "함수"를 도구로 쓸 준비가 되었습니다.

4.3 YOLO 의 진화 — v1 부터 v8 까지 핵심 변화

YOLO 는 "You Only Look Once"의 약자입니다. 2015 년 v1 이 발표되었을 때의 충격은 컸습니다. 그때까지 Detection 은 "사진을 여러 영역으로 잘라 각각 분류하는" 방식이라 느렸는데(R-CNN, Fast R-CNN), YOLO 는 사진 한 장을 한 번만 신경망에 통과시켜 모든 객체를 한꺼번에 찾아냈습니다 — 그래서 이름이 "한 번만 본다" 입니다.

이후 v2, v3, v4, v5 를 거치며 정확도와 속도가 꾸준히 개선되었습니다. v4 는 정확도 측면의 큰 도약이었고, v5 는 PyTorch 기반으로 옮겨오면서 사용성이 대폭 좋아졌습니다. 현재 가장 자주 쓰이는 버전은 Ultralytics 사가 2023 년 발표한 YOLOv8 입니다.

YOLOv8 의 특징:

- PyTorch 기반, 사용이 매우 쉽다 (`model = YOLO('yolov8n.pt')` 한 줄로 로드).
- Detection 뿐 아니라 Segmentation, Pose Estimation, Classification 까지 같은 API 로 지원.
- Nano(n)부터 Small(s), Medium(m), Large(l), Extra-large(x)까지 5 가지 크기 옵션.

- ONNX, TensorRT, CoreML 등 다양한 형식으로 export 가능.

우리는 라즈베리파이에서 돌릴 것이므로 가장 작은 YOLOv8n 을 씁니다. 320 만 개 가중치, 모델 파일 약 6.2MB, COCO mAP@0.5 약 0.52.

4.4 사전 학습 모델로 80 개 객체 인식하기 (COCO 데이터셋)

YOLOv8n 은 COCO(Common Objects in Context)라는 데이터셋으로 미리 학습되어 있습니다. COCO 에는 80 가지 일상 객체가 포함됩니다 — person, bicycle, car, dog, cat, horse, chair, bottle 등. 다행히 "cat"이 80 개 중에 있어서, 우리집 고양이도 처음에는 "cat" 클래스로 인식됩니다.

```
from ultralytics import YOLO

# 처음 호출 시 인터넷에서 자동 다운로드 (약 6MB)
model = YOLO("yolov8n.pt")

# 추론
results = model("cat.jpg")

# 결과 파싱
for r in results:
    for box in r.boxes:
        cls_id = int(box.cls[0])
        cls_name = model.names[cls_id]
        conf = float(box.conf[0])
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        print(f"{cls_name} {conf:.2f} ({x1:.0f},{y1:.0f})-({x2:.0f},{y2:.0f})")

# 결과 이미지 저장 (박스가 그려진)
results[0].save("detected.jpg")
```

이 짧은 코드가 — 그 모든 신경망 이론의 결과물입니다. 한 줄로 사전학습 모델을 로드하고, 또 한 줄로 추론합니다. 우리는 4.2 에서 그 "안에서 무슨 일이 일어나는지"를 이미 이해했습니다.

4.5 라즈베리파이에서 YOLOv8 실행하기 — FPS 와 최적화

위의 코드를 그대로 라즈베리파이에 가져와 실시간 카메라 영상에 적용하면, Pi 5

에서는 4~6 FPS, Pi 4에서는 1~2 FPS 정도가 나옵니다. 차이가 큼니다.

FPS를 끌어올리는 데는 여러 가지 방법이 있습니다.

- 입력 해상도 줄이기 — 640x480에서 320x240으로 줄이면 약 2배 빨라짐 (작은 객체 인식 정확도는 약간 저하).
- 모델을 ONNX 또는 NCNN으로 변환 — PyTorch 런타임의 오버헤드 제거. 약 1.5~2배.
- INT8 양자화 — 가중치를 32비트 부동소수점 → 8비트 정수로 변환. 약 2~3배.
- 매 프레임 추론하지 않기 — N 프레임마다 한 번씩만 YOLO, 그 사이는 추적 알고리즘(예: ByteTrack)으로 가벼운 위치 갱신.
- Coral USB Accelerator 같은 외장 NPU 사용 — 약 5~10배 (단, 비용 추가).

이 중 ONNX 변환과 양자화는 6장에서 파인튜닝한 모델을 배포할 때 같이 다룹니다. 매 프레임 추론하지 않는 "추적 + 검출 분리" 패턴은 8장의 통합 시스템에서 다룹니다.

4.6 색상 검출 vs YOLO — 무엇이 다른가

3장에서 HSV로 파란공을 검출했고, 이번 장에서 YOLO로 고양이를 검출했습니다. 두 방법의 차이를 정리해 둘 만합니다.

항목	HSV 색상 검출	YOLO Detection
무엇을 보는가	픽셀의 색깔만	픽셀의 패턴 전체
견고함	조명·각도에 약함	학습 데이터 범위에서 견고
대상의 종류	색이 뚜렷한 단순 물체	일상의 거의 모든 객체
실행 속도	매우 빠름 (수백 FPS)	상대적으로 느림 (수~수십 FPS)
유연성	새 대상 = 임계값 새로	새 대상 = 데이터+학습 필요
"내 고양이"	불가능	파인튜닝하면 가능
필요 자원	CPU 만	GPU 권장 (학습 시)
설명 가능성	높음 (어떤 색인지 명시)	낮음 (왜 그 답인지 불투명)

Vol.4가 "파란공 → COCO 고양이 → 우리집 고양이"라는 3단계 학습 곡선을 따라가는 이유가 이 표에 들어 있습니다. 단순한 도구로 시작해, 한계를 발견하고, 더 강력한 도구로 넘어가는 흐름입니다.

엔지니어의 판단:

두 방법은 "한쪽이 더 좋다"가 아니라 "용도가 다르다"입니다. 실제 산업 현장에서도 색상 검출이 충분한 곳(예: 라벨 색 검사기, 컨베이어 위 빨강·노랑 분류)에서는 — YOLO를 안 씁니다. "가장 단순한 도구로 충분한 일"에 신경망을 갖다 대는 것은 — 임베디드 자원과 전력의 낭비입니다.

4.7 COCO 모델의 한계 — 옆집 고양이도 우리집 고양이?

자, 이제 솔직해질 시간입니다. YOLOv8n + COCO 로 "고양이"를 인식할 수 있다 — 그래서요? 이게 우리 RC 카에 정말 충분합니까?

문제는 두 가지입니다.

문제 ①: 내 고양이 vs 옆집 고양이를 구분할 수 없다

COCO 모델은 모든 고양이를 "cat"으로 통합 인식합니다. 우리집 RC 카가 "고양이"만 따라다닌다면, 동네 길고양이가 시야에 들어오는 순간 추적 대상이 바뀝니다.

이게 왜 문제일까요? 시나리오를 생각해 봅시다. 거실에서 우리집 검은 고양이 "미요"를 추적 중인데, 창문 밖에서 옆집 흰 고양이 "치즈"가 지나갑니다. RC 카는 둘 다 "cat"으로 인식하고, 더 큰 박스(가까운 쪽)를 따라갑니다. 만약 치즈가 더 가까이 있다면 — RC 카는 우리집 고양이를 두고 옆집 고양이를 쫓아갑니다.

"우리집 고양이만" 추적하려면 — 우리집 고양이의 사진을 모아 학습시켜야 합니다. 즉, 새로운 클래스 "my_cat"을 만들고 그 클래스만 인식하도록 모델을 가르쳐야 합니다.

문제 ②: 누워 있거나 옆모습인 고양이는 잘 못 본다

COCO 데이터셋의 고양이 사진은 대부분 "카메라를 정면으로 보는 고양이"입니다. 인터넷에서 "cat"으로 검색해 나오는 사진들이 그렇습니다. 그래서 우리집 고양이가 옆으로 누워 있거나, 등을 보이거나, 박스 안에 웅크리고 있으면 — YOLO가 놓칠 수 있습니다.

이것 역시 — 우리집 고양이의 다양한 자세를 학습시켜야 합니다. 우리 환경에서 자주 나오는 자세들을 데이터로 모아, 모델에게 "이런 자세도 cat이다"를 가르치는 것입니다.

결국 — 파인튜닝이 답이다

이 두 문제가 5 장(자동 라벨링)과 6 장(파인튜닝)으로 우리를 끌고 갑니다. "마지막 1 퍼센트를 위한 작업"이 아닙니다 — 우리집 환경에서 실용적인 추적 시스템을 만들기 위해서는 반드시 거쳐야 하는 과정입니다.

AI 협업의 진짜 가치:

이 단계는 AI에게 "COCO 모델로 우리집 고양이를 구분할 수 있나?" 라고 물으면 솔직하게 알려줍니다. AI는 자기가 잘하는 것뿐 아니라 자기가 못하는 것도 잘 압니다. 시스템 설계의 한계를 발견하는 단계에서 AI는 좋은 토의 상대가 됩니다 — 마치 경험 많은 동료 개발자처럼.

4.8 AI와 함께하는 YOLO 추론 실습

3 장의 파란공 검출 코드를 "고양이 검출 코드"로 바꿔 봅시다. AI와 함께 작업합니다.

AI와 함께 설계하기

3 장에서 만든 파란공 검출 코드(picamera2 + OpenCV)를 베이스로, 이번에는 YOLOv8n 으로 'cat' 클래스만 검출하는 코드를 만들어 줘. 요구사항: 1) picamera2 로 640x480, 10 FPS 2) ultralytics YOLO('yolov8n.pt')로 추론 3) 'cat' 클래스만 표시 (COCO 에서 cat 은 클래스 ID 15) 4) 신뢰도가 0.5 이상인 박스만 5) 가장 큰 cat 한 마리만 추적 대상으로 선택, 중심에 빨간 점 표시 6) FPS, 추적 대상의 좌우 위치(LEFT/CENTER/RIGHT), 신뢰도를 화면에 표시 7) 'q' 키로 종료, 종료 시 카메라 자원 안전하게 해제 전체 코드와 함께, 라즈베리파이에서 FPS 를 높이기 위한 최적화 팁 3 가지도 알려줘.

AI가 만든 코드를 실행하고, 우리집 고양이를 카메라 앞에 데려와 보세요. 인식이 잘 되는 자세와 안 되는 자세를 직접 확인합니다. "왜 옆모습일 때 못 잡지?" 같은 의문을 — 직접 확인하는 단계가 5 장의 데이터 수집 전략으로 자연스럽게 이어집니다.

AI와 함께 설계하기

위 YOLO 추론 코드를 라즈베리파이 4 에서 돌렸더니 FPS 가 2~3 밖에 안 나온다. 다음 최적화 옵션 중 어떤 조합이 가장 효과적일지 분석해 줘: 1) 입력 해상도 640x480 → 320x240 2) 모델을 ONNX 로 변환 3) INT8 양자화 4) 3 프레임마다 한 번씩만 추론, 나머지는 이전 박스를 그대로 표시 각 옵션의 예상 FPS 향상, 정확도 영향, 구현

난이도를 표로 정리해 줘.

이 코드가 동작하는 순간 — 우리 RC 카의 "눈"이 완성됩니다. 그러나 4.7에서
짚었듯 옆집 고양이가 와도 똑같이 "cat"으로 잡힙니다. 다음 장에서 이 문제를
풀어갑니다.

Chapter 4 핵심 정리

Detection = 무엇이 어디에 있는지 — RC 카에는 Detection 이 답이다.

신경망은 마법이 아니라 "가중치 곱하고 더하고 ReLU"의 깊은 반복일 뿐이다.

층을 쌓을수록 픽셀 → 엣지 → 부품 → 객체 순으로 추상도가 올라간다.

CNN 은 같은 필터를 사진 전체에 슬라이딩해 위치 불변성을 얻는다.

학습 = 손실을 줄이는 방향으로 가중치를 조금씩 바꾸는 일 (수천만 번).

COCO YOLOv8n 은 80 개 객체를 인식하지만, '우리집 고양이'는 구분 못한다 — 5 장/6
장의 동기.

라즈베리파이에서 YOLO 최적화 = 해상도 ↓ + ONNX + INT8 + 추적 분리.

Chapter 5. 자동 라벨링 — 데이터셋을 스스로 만드는 RC 카

4 장에서 우리는 "우리집 고양이만 인식하려면 직접 학습해야 한다"는 결론에 달았습니다. 학습의 출발점은 데이터셋입니다 — "우리집 고양이가 박힌 사진"과 "그 사진에서 고양이의 좌표"를 짝지어 모은 모음. 이 챕터에서 우리는 그 데이터셋을 — 너무 힘들이지 않고 — 만드는 법을 배웁니다.

5.1 왜 직접 학습이 필요한가 — 우리집 고양이는 COCO 에 없다

4 장에서 본 COCO 모델의 한계는 분명합니다. "우리집 고양이만" 추적하려면 — 우리집 고양이의 사진을 모아 모델에게 보여줘야 합니다. 이것이 파인튜닝(Fine-tuning)입니다.

파인튜닝의 첫 단계는 데이터셋입니다. 사진은 카메라로 찍으면 되지만, 좌표 — "이 사진의 (350, 200)부터 (520, 400)까지가 고양이입니다"라는 정보 — 는 누가 만들까요?

전통적으로는 사람이 마우스로 박스를 그렸습니다. 사진 한 장에 30 초~1 분. 1,000 장이면 8~17 시간. 끔찍합니다. 실제로 자율주행차 회사에서 라벨링 외주 비용이 수십억원에 달하는 이유가 이것입니다.

Vol.4 의 트릭은 — COCO YOLO 를 이용해 1 차 라벨링을 자동으로 시키는 것입니다. 어차피 COCO 모델은 "고양이"를 일반적으로 잘 찾으니까요. 모든 자세에서 완벽하지는 않지만, 충분히 많은 사진에서 정확하게 박스를 칩니다. 사람은 검수만 합니다 — "맞다 / 틀렸다"만 보고, 틀린 박스만 수정합니다. 처음부터 그리는 것보다 8~10 배 빠릅니다.

AI 시대의 데이터 작업:

이 "AI 가 만든 1 차 라벨을 사람이 검수" 패턴이 — 현재 산업계의 표준입니다. Tesla, Waymo 같은 회사들도 raw 데이터 라벨링은 모두 AI 1 차 + 사람 검수 방식을 씁니다. 우리가 하는 일은 그 패턴의 가장 단순한 형태입니다.

5.2 데이터 수집 전략 — 다양한 각도, 조명, 거리, 자세

데이터셋의 양도 중요하지만, 다양성이 더 중요합니다. 100 장의 정면 얼굴 사진보다

30 장의 다양한 각도가 더 좋은 모델을 만듭니다. 이건 통계적 직관입니다 — 학습 데이터에 없는 변형(variation)은 모델이 본 적 없는 상황이고, 결과를 예측할 수 없습니다.

우리집 고양이를 찍을 때 의도적으로 변주를 줍니다.

- 각도 — 정면, 옆모습, 뒤에서, 위에서. 적어도 네 각도 모두 포함.
- 자세 — 서 있는 모습, 앉은 모습, 누운 모습, 웅크린 모습, 점프 중.
- 거리 — 가까이(50cm), 중간(2m), 멀리(5m). RC 카가 실제 운용할 거리 범위를 모두 포함.
- 조명 — 낮·저녁·밤, 거실·부엌·복도. 같은 위치라도 시간대를 바꿈.
- 배경 — 바닥, 소파 위, 박스 안, 캣타워. 배경이 "고양이의 일부"로 학습되지 않도록 다양하게.
- 부분 가림(occlusion) — 가구 뒤에서 머리만 보이는 경우, 다리만 보이는 경우. 실제 추적에서 자주 일어남.

실용적으로는 — RC 카에 카메라를 장착하고 거실을 돌아다니며 영상 30 초씩 5~10 개를 찍는 것이 빠릅니다. 영상에서 0.5 초 간격으로 프레임을 추출하면 200~400 장의 사진이 한 번에 모입니다.

```
import cv2
import os

cap = cv2.VideoCapture("cat_video.mp4")
fps = cap.get(cv2.CAP_PROP_FPS)
interval = int(fps * 0.5) # 0.5 초 간격
os.makedirs("dataset/images", exist_ok=True)

i = 0
n = 0
while True:
    ret, frame = cap.read()
    if not ret:
        break
    if i % interval == 0:
        cv2.imwrite(f"dataset/images/frame_{n:04d}.jpg", frame)
        n += 1
    i += 1
```

```
cap.release()
print(f"{n}장 추출 완료")
```

얼마나 모아야 하는가

정답은 없지만 경험적 기준은 있습니다.

- 최소 — 100~200 장. 처음 시도해 볼 수 있는 수준. 정확도는 낮을 수 있음.
- 권장 — 300~500 장. 실용적 정확도가 나오는 수준.
- 충분 — 1,000 장 이상. 거의 모든 상황에서 잘 동작.

그리고 다양성이 양보다 중요합니다 — 같은 자세의 1000 장보다 다양한 자세의 300 장이 낫습니다.

5.3 사전 학습 YOLO 로 1 차 라벨링 자동화

이제 모은 사진들에 대해 COCO 모델을 돌려, "cat 클래스의 바운딩 박스"를 자동으로 뽑아냅니다. 이 결과를 YOLO 학습용 라벨 포맷으로 저장합니다.

YOLO 라벨 포맷

Ultralytics YOLO의 라벨 포맷은 사진마다 같은 이름의 .txt 파일을 만드는 방식입니다.

```
# dataset/images/frame_0001.jpg 에 대응하는
# dataset/labels/frame_0001.txt 의 내용:
0 0.523 0.412 0.265 0.378

# 형식: <class_id> <x_center> <y_center> <width> <height>
# 좌표와 크기는 모두 0~1로 정규화한 값 (이미지 크기로 나눈 비율)

# 한 사진에 객체가 여러 개면 한 줄씩:
0 0.523 0.412 0.265 0.378
0 0.215 0.621 0.180 0.220
```

"정규화한 좌표"를 쓰는 이유는 — 학습 도중 사진을 다른 해상도로 리사이즈해도 라벨이 자동으로 맞기 때문입니다. 640x480에서 그렸든 1920x1080에서 그렸든

— 비율은 같습니다.

자동 라벨링 코드

```
from ultralytics import YOLO
from pathlib import Path
import cv2

model = YOLO("yolov8n.pt")
img_dir = Path("dataset/images")
lbl_dir = Path("dataset/labels")
lbl_dir.mkdir(exist_ok=True)
CAT_CLASS_ID = 15 # COCO 에서 cat 은 15 번

for img_path in sorted(img_dir.glob("*.jpg")):
    img = cv2.imread(str(img_path))
    h, w = img.shape[:2]
    results = model(img, verbose=False)[0]

    lines = []
    for box in results.boxes:
        if int(box.cls[0]) == CAT_CLASS_ID and float(box.conf[0]) > 0.5:
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            xc = (x1 + x2) / 2 / w
            yc = (y1 + y2) / 2 / h
            bw = (x2 - x1) / w
            bh = (y2 - y1) / h
            # 새 클래스 ID 는 0 (우리집 고양이만 한 클래스)
            lines.append(f"0 {xc:.6f} {yc:.6f} {bw:.6f} {bh:.6f}")

    out_path = lbl_dir / (img_path.stem + ".txt")
    out_path.write_text("\n".join(lines))

print(f"{len(list(img_dir.glob('*.jpg')))}장 라벨링 완료")
```

이 코드 한 번 실행으로 — 300 장이 약 5 분 안에 라벨링됩니다. (Pi 에서 돌릴 필요 없습니다. PC 에서 더 빠르게.) 5 장의 핵심 시간 절약이 여기서 일어납니다.

5.4 라벨 검수 도구 사용법 (CVAT 또는 Label Studio)

자동 라벨링은 완벽하지 않습니다. COCO 모델이 놓치는 자세도 있고, 잘못 찾는 경우도 있습니다. 예를 들어 — 강아지 인형을 "cat"으로 잘못 인식하거나, 누워있는 고양이를 "잘 안 보여서" 놓치는 경우 등. 그래서 사람이 검수해야 합니다.

CVAT

Computer Vision Annotation Tool. 웹 기반, 기능이 풍부합니다. Docker 로 띄우거나, 클라우드 버전(cvat.ai)을 무료로 쓸 수 있습니다.

- 장점 — 자동 라벨링 import 지원, 단축키가 풍부해서 빠르게 작업, 다중 사용자 협업.
- 단점 — 설치가 약간 복잡, UI가 처음에는 낯섭.

Label Studio

설치가 더 간단하고 UI가 깔끔합니다.

```
pip install label-studio
label-studio start
# 브라우저로 http://localhost:8080 접속
```

- 장점 — 설치 한 줄, UI 직관적, AI 기반 자동화 옵션 다양.
- 단점 — 키보드 단축키가 CVAT 보다 적음.

검수 작업 흐름

어느 쪽이든 작업 흐름은 같습니다.

- ① 자동 라벨이 적용된 데이터셋을 import.
- ② 한 장씩 보며 박스가 잘 잡혔는지 확인.
- ③ 정확하면 → 다음으로 (단축키, 보통 →).
- ④ 어긋난 박스 → 드래그로 조정.
- ⑤ 박스가 없는데 고양이가 있으면 → 직접 그리기.
- ⑥ 박스가 있는데 고양이가 아니면 → 삭제.
- ⑦ 모두 끝나면 YOLO 포맷으로 export.

체감 시간:

300 장 데이터셋 기준으로, 순수 수동 라벨링은 4~5 시간 걸리지만, 자동 라벨링 +

검수는 30~40 분이면 끝납니다. 시간을 8~10 배 줄여줍니다. 이 "AI가 1 차 작업, 사람이 검수"의 분업이 — 데이터 작업의 표준 패턴입니다.

5.5 YOLO 데이터셋 포맷 이해 (images/, labels/, data.yaml)

Ultralytics YOLO 학습에 쓰는 데이터셋의 표준 구조는 다음과 같습니다.

```
my_cat/
├── images/
│   ├── train/
│   │   ├── frame_0001.jpg
│   │   ├── frame_0002.jpg
│   │   └── ...
│   └── val/
│       ├── frame_0250.jpg
│       └── ...
├── labels/
│   ├── train/
│   │   ├── frame_0001.txt
│   │   ├── frame_0002.txt
│   │   └── ...
│   └── val/
│       ├── frame_0250.txt
│       └── ...
└── data.yaml
```

images/와 labels/의 폴더 구조가 정확히 대칭이어야 합니다. 각 이미지 파일과 같은 이름의 라벨 파일이 짝을 이루어야 합니다 — frame_0001.jpg 와 frame_0001.txt 처럼. 짝이 안 맞으면 YOLO 가 "라벨이 없다"고 보고 학습에서 제외합니다.

data.yaml — 데이터셋의 메타정보

data.yaml 은 데이터셋이 어디 있고 클래스가 무엇인지를 정의하는 작은 설정 파일입니다.

```
# data.yaml
path: /home/pi/my_cat      # 데이터셋 루트 (절대 경로 권장)
train: images/train        # 학습 이미지 폴더 (path로부터의 상대 경로)
```

```
val: images/val          # 검증 이미지 폴더
nc: 1                    # 클래스 개수 (우리는 cat 하나만)
names:
  0: my_cat              # 클래스 ID 와 이름
```

우리집 고양이 외에 우리집 강아지도 추적하고 싶다면 다음과 같이 확장합니다.

```
nc: 2
names:
  0: my_cat
  1: my_dog
```

그러면 라벨 파일도 클래스 ID 를 구분해서 적습니다 (0 이면 cat, 1 이면 dog).

train/val 분할 비율

학습용(train)과 검증용(val)을 어떻게 나눠야 할까요? 일반적인 권장 비율은 — train 80%, val 20%입니다. 300 장이라면 train 240 장, val 60 장. val 데이터는 학습에 쓰지 않고 "모델이 본 적 없는 사진"으로 성능을 평가하는 데만 씁니다.

주의 — train 과 val 에 "같은 영상에서 나온 비슷한 프레임"이 섞이지 않도록 해야 합니다. 그러면 모델이 "외운 것"을 "맞춘 것"으로 착각해 — 평가 점수가 비현실적으로 높게 나옵니다. 영상 단위로 train/val 을 나누는 것이 더 안전합니다.

5.6 AI 와 함께하는 자동 라벨링 파이프라인

위에서 본 모든 단계 — 영상에서 프레임 추출, COCO 라벨링, 데이터셋 분할 — 를 한 번에 처리하는 파이프라인을 AI 와 함께 만들어 봅시다.

AI 와 함께 설계하기

다음 작업을 한 번에 처리하는 Python 스크립트를 만들어 줘: 1) 'videos/' 폴더 안의 모든 mp4 에서 0.5 초 간격으로 프레임 추출 2) 추출한 모든 사진에 YOLOv8n 으로 자동 라벨링 (cat 클래스만, 신뢰도 0.5 이상) 3) 라벨이 한 개 이상 있는 사진만 살리고, 빈 사진은 제거 (학습에 노이즈가 됨) 4) train/val 을 8:2 로 랜덤 분할 (단, 같은 영상에서 나온 프레임은 한쪽에만 가도록) 5) my_cat/ 폴더 구조로 출력 + data.yaml 자동 생성 6) 마지막으로 통계 출력 (총 사진 수, train/val 수, 라벨 박스 수, 영상별 분포) 스크립트와 함께, 데이터셋의 품질을 사전 점검할 수 있는 시각화 팁(예: 박스 크기 분포 히스토그램)

도 알려줘.

AI가 만든 파이프라인은 일종의 "데이터 공장"입니다. 이 단계가 잘 만들어지면, 다른 객체(개, 사람, 특정 장난감 등)로 바꾸는 것도 클래스 ID 한 줄만 수정하면 됩니다. "확장 가능한 파이프라인"을 만드는 것이 — AI 시대 임베디드 개발자의 좋은 습관입니다.

AI와 함께 설계하기

위 파이프라인을 돌렸더니 train에 240 장, val에 60 장이 만들어졌다. 그런데 val의 일부 사진을 보니 — 학습에 쓰면 안 될 것 같은 사진들이 보인다. 예를 들면 고양이가 너무 작거나(전체 화면의 5% 미만), 모션 블러가 심하거나, 박스가 잘못 그려진 사진들. 이런 "품질 낮은 샘플"을 자동으로 걸러내는 추가 스크립트를 만들어 줘. 어떤 기준으로 필터링할지 함께 설명해 줘.

Chapter 5 핵심 정리

데이터셋의 양보다 다양성 — 각도·조명·자세·거리를 의도적으로 변주한다.

RC카로 영상을 찍고 0.5 초 간격 프레임 추출하면 수백 장이 30 분 만에 모인다.

COCO YOLO로 1차 자동 라벨링 → 사람은 검수만 — 시간을 8~10배 줄인다.

Ultralytics YOLO 표준 구조: images/{train,val} + labels/{train,val} + data.yaml.

train/val 비율은 8:2가 일반적. 같은 영상의 프레임이 양쪽에 섞이지 않도록 주의.

자동 라벨링 + 검수 + 분할의 전체 파이프라인을 한 스크립트로 만들어 두면 — 다른 객체로 확장 가능.

AI에게 데이터 품질 필터링 스크립트까지 부탁하면 — 노이즈를 줄여 학습 결과가 좋아진다.

Chapter 6. 파인 튜닝 — 우리집 고양이만 알아보는 YOLO (고양이 2 차 도전)

드디어 우리집 고양이를 "인격적으로" 알아보는 모델을 만들 차례입니다. 5 장에서 데이터셋을 준비했으니, 이제 그 데이터로 YOLO 를 학습시킵니다. 그런데 "학습"이라는 단어가 무겁게 들릴 수 있습니다 — "수백만 장 데이터로 며칠씩 GPU 를 돌려야 하는 일 아닌가?" 다행히 우리는 "파인튜닝"이라는 지름길을 씁니다.

6.1 전이학습(Transfer Learning) 개념과 원리

우리집 고양이 사진 300 장으로 YOLO 를 "처음부터" 학습시키면 어떨까요? 결과는 처참합니다. YOLO 같은 모델을 처음부터 학습시키려면 보통 수십만~수백만 장이 필요합니다. 우리 300 장으로는 — 모델이 "고양이"라는 개념조차 형성하지 못합니다.

그래서 우리는 다른 방법을 씁니다 — 이미 잘 학습된 모델을 가져와서, 우리 데이터에 맞게 "미세하게" 조정합니다. 이것이 전이학습(Transfer Learning)이고, 그 한 형태가 파인튜닝(Fine-tuning)입니다.

왜 이게 가능한가

직관은 4.2 절에서 본 신경망의 계층 구조에서 옵니다.

- 초반 층 — 픽셀 → 엣지 → 모서리. 어떤 객체든 공통적으로 쓰이는 저수준 특징.
- 후반 층 — 모서리 → 부품 → 객체. 클래스별로 특화되는 고수준 특징.

초반 층은 COCO 에서 학습한 것을 그대로 써도 충분합니다. 고양이의 엣지든 차의 엣지든 — 같은 종류의 시각 특징이니깐요. 우리는 후반 층 위주로 "우리집 고양이"에 맞게 조금만 조정하면 됩니다. 그래서 300 장 같은 작은 데이터로도 가능해집니다.

사람의 학습과 비교:

이건 사람이 새 언어를 배울 때와 비슷합니다. 한국어를 잘하는 사람이 일본어를 배울 때 — 한국어 학습으로 만든 "언어 회로"의 많은 부분을 재사용합니다 (문법 구조, 학습 전략, 어휘 암기 기법). 그래서 새 언어를 어린 시절보다 훨씬 빨리 배웁니다. 전이학습이 정확히 같은 메커니즘입니다.

6.2 학습 환경 선택 — PC GPU vs Google Colab

라즈베리파이로 학습하면 안 됩니다. CPU 만으로는 며칠이 걸립니다. 학습은 GPU 가 있는 환경에서 하고, 결과 모델을 라즈베리파이에 올리는 흐름입니다.

PC GPU

NVIDIA GPU(RTX 3060 이상)가 있다면 가장 빠르고 통제 가능합니다.

- 장점 — 빠른 학습(YOLOv8n + 300 장 + 100 epoch = 약 10~20 분), 데이터 보안, 인터넷 끊겨도 작업 가능.
- 단점 — 하드웨어 비용 부담, CUDA/PyTorch 설치가 처음에는 까다로움.

설치 명령:

```
# CUDA 12.1 기준
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
pip install ultralytics

# 설치 검증
python -c "import torch; print(torch.cuda.is_available())"
# True 가 나와야 함
```

Google Colab

무료 GPU(T4)를 받을 수 있어 별도 하드웨어 없이 학습 가능합니다.

- 장점 — 무료, 설치 불필요, 어디서나 접속 가능.
- 단점 — 세션 시간 제한(무료 버전), 데이터를 매번 업로드(또는 Google Drive 마운트).

Colab 에서 학습하는 노트북의 첫 셀:

```
# GPU 확인
!nvidia-smi

# Google Drive 마운트
from google.colab import drive
drive.mount('/content/drive')

# 데이터셋 위치 확인
!ls /content/drive/MyDrive/my_cat/
```

```
# Ultralytics 설치
!pip install ultralytics
```

개인용 GPU가 없다면 Colab을 추천합니다. 동일한 학습이 약 15~25분 걸립니다.

6.3 데이터셋 분할과 학습 파라미터

5장에서 만든 데이터셋이 my_cat/에 준비되었다고 가정합니다. 학습 코드는 놀랄만큼 짧습니다.

```
from ultralytics import YOLO

# 사전학습 모델로 시작 (COCO에서 학습된 가중치)
model = YOLO("yolov8n.pt")

# 우리 데이터로 파인튜닝
results = model.train(
    data="my_cat/data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    name="my_cat_v1",
    patience=20,
)

# 결과는 runs/detect/my_cat_v1/ 폴더에 저장됨
```

주요 파라미터의 의미

- epochs=100 — 전체 데이터셋을 100번 반복 학습. 작은 데이터셋에서는 50~150이 적당.
- imgsz=640 — 학습 입력 해상도. 작을수록 빠르지만 작은 객체 인식이 약해진다. RC카 운용 거리를 고려해 결정.
- batch=16 — 한 번에 처리하는 사진 수. GPU 메모리에 맞춰 조정. 메모리 부족 시 batch=8 또는 4로.
- patience=20 — 20 epoch 동안 개선이 없으면 학습 중단 (early stopping). 과적합 방지.

- name='my_cat_v1' — 결과 폴더 이름. v2, v3 로 실험할 때 구분 용이.

Data Augmentation — 데이터를 "가짜로 늘리는" 기법

Ultralytics YOLO 는 학습 도중 자동으로 데이터 증강(augmentation)을 합니다 — 같은 사진을 회전·뒤집기·색조 변경·일부 가림 등으로 다양하게 변형해 학습. 300 장이 사실상 수천 장 효과를 냅니다.

```
# 기본 augmentation 외에 직접 조절도 가능
model.train(
    data="my_cat/data.yaml",
    epochs=100,
    hsv_h=0.015, # 색조 변경 정도
    hsv_s=0.7,   # 채도 변경
    hsv_v=0.4,   # 명도 변경
    fliplr=0.5,  # 좌우 뒤집기 확률
    flipud=0.0,  # 상하 뒤집기 (고양이는 보통 위/아래가 의미 있으므로 0)
)
```

6.4 학습 결과 평가 — mAP, Precision, Recall, Confusion Matrix

학습이 끝나면 runs/detect/my_cat_v1/ 폴더에 결과들이 저장됩니다. 그중 살펴봐야 할 것들이 있습니다.

주요 지표

- Precision — 모델이 "고양이"라고 한 박스 중 실제로 고양이인 비율. 높을수록 오탐(false positive)이 적음. RC 카 입장에서 — "엉뚱한 것을 쫓아가지 않는다" 척도.
- Recall — 실제 고양이 박스 중 모델이 찾은 비율. 높을수록 누락(false negative)이 적음. RC 카 입장에서 — "고양이를 놓치지 않는다" 척도.
- mAP@0.5 — 박스가 정답과 50% 이상 겹치면 맞춤으로 칠 때의 평균 정밀도. 0.8 이상이면 매우 좋다. 0.6 미만이면 다시 학습.
- mAP@0.5:0.95 — 더 엄격한 평가(여러 IoU 임계값 평균). 0.5 이상이면 양호, 0.6 이상이면 우수.

그래프 — results.png

학습 진행 중에 손실(loss)이 줄어드는 그래프와 mAP가 올라가는 그래프가 자동으로 그려집니다. results.png를 열어 다음을 확인합니다.

- box_loss, cls_loss, dfl_loss가 epoch이 진행되며 줄어드는가? — 정상.
- val 손실이 어느 시점부터 다시 올라가기 시작하는가? — 과적합 신호.
- mAP가 어느 epoch 즈음에서 수렴(평평해짐)했는가? — 그 이후 학습은 시간 낭비.

과적합(Overfitting)의 직관:

수능 모의고사로 비유하자면, 모델이 학습 데이터를 '외운' 상태입니다. 학습 데이터에서는 잘 맞추지만, 본 적 없는 새 사진에서는 못 맞춥니다. 데이터가 너무 적거나, 학습을 너무 오래 시켰을 때 일어납니다. 해결책: 더 많은 데이터, 더 강한 augmentation, 더 일찍 학습 중단(early stopping).

Confusion Matrix

confusion_matrix.png는 클래스를 잘 구분하는지 보여주는 행렬입니다. 우리는 클래스가 하나(my_cat)뿐이라 단순하지만, 만약 my_cat과 my_dog 두 클래스라면 — 둘을 헷갈리는 경우가 얼마나 되는지 시각적으로 확인할 수 있습니다.

6.5 모델 경량화 — INT8 양자화와 ONNX 변환

학습된 모델은 best.pt 파일로 저장됩니다. 이 파일을 그대로 라즈베리파이에 올리면 동작하긴 하지만, 느립니다. 경량화 단계가 필요합니다.

ONNX 변환

ONNX(Open Neural Network Exchange)는 모델 표현의 표준 형식입니다.

PyTorch 런타임의 무거운 오버헤드를 제거하고, 더 가벼운 런타임(onnxruntime)으로 추론합니다. 보통 1.5~2배 빨라집니다.

```
from ultralytics import YOLO

model = YOLO("runs/detect/my_cat_v1/weights/best.pt")
model.export(format="onnx")  # best.onnx 생성
```

```
# 라즈베리파이에 best.onnx 를 복사한 뒤:  
model = YOLO("best.onnx", task="detect")  
# 이후 사용법은 .pt 와 동일
```

INT8 양자화 (Quantization)

기본 모델은 가중치를 32 비트 부동소수점으로 저장합니다. 이걸 8 비트 정수로 줄이면 — 모델 크기가 1/4, 추론 속도가 2~3 배가 됩니다. 정확도가 약간(보통 1~3% mAP) 떨어지지만, 라즈베리파이에서는 받을 만한 거래입니다.

```
model.export(format="onnx", int8=True, data="my_cat/data.yaml")  
# int8 양자화에는 calibration 데이터가 필요해서 data 를 전달
```

양자화의 원리는 — "32 비트로 표현하던 가중치 범위(예: -3.14 ~ 5.27)를 8 비트 정수 256 단계(0~255)에 매핑하는 것"입니다. 표현력은 줄지만, 신경망의 robustness 덕분에 결과 품질은 의외로 크게 떨어지지 않습니다.

NCNN — 라즈베리파이 전용 최적화

NCNN 은 Tencent 에서 만든 모바일/임베디드 전용 추론 엔진입니다. ARM CPU 의 NEON SIMD 를 최대한 활용해 — 라즈베리파이에서 ONNX 보다 빠른 경우가 많습니다.

```
model.export(format="ncnn")  
# best_ncnn_model/ 폴더 생성  
  
model = YOLO("best_ncnn_model", task="detect")
```

6.6 라즈베리파이에 커스텀 모델 올리기

변환한 모델을 라즈베리파이에 복사하고, 4 장의 추론 코드에서 모델 경로만 바꿔주면 됩니다.

```
# PC 에서 라즈베리파이로 복사  
scp best.onnx pi@rccar.local:~/rccar/
```

```
# 또는 ncnn 폴더 전체 복사
scp -r best_ncnn_model/ pi@rccar.local:~/rccar/

# 라즈베리파이에서 추론
from ultralytics import YOLO
model = YOLO("/home/pi/rccar/best.onnx", task="detect")

results = model("test_cat.jpg")
for r in results:
    for box in r.boxes:
        cls_id = int(box.cls[0])
        cls_name = model.names[cls_id] # 'my_cat'
        conf = float(box.conf[0])
        print(f"{cls_name} {conf:.2f}")
```

이 시점에서 우리 RC 카는 — 동네 길고양이를 무시하고 우리집 고양이만 정확히 인식할 수 있습니다. 5 장과 6 장의 모든 수고가 — 이 한 가지 능력을 위한 것입니다.

6.7 AI 와 함께하는 파인 튜닝 실습

학습 결과를 해석하는 단계가 사실 가장 어렵습니다. mAP 숫자만으로는 "우리 모델이 실용적인지" 알기 어렵습니다. AI 에게 결과 해석을 부탁하면 좋습니다.

AI 와 함께 설계하기

YOLOv8n 을 우리집 고양이 데이터셋(train 240 장, val 60 장)으로 100 epoch 학습시킨 결과는 다음과 같다: - mAP@0.5 = 0.87 - mAP@0.5:0.95 = 0.62 - Precision = 0.91, Recall = 0.84 - val box_loss 가 epoch 80 이후 살짝 다시 오르기 시작함 - 옆모습 고양이를 잘 못 잡음 (육안으로 확인) 이 결과를 어떻게 해석해야 할까? 실제 RC 카에 올렸을 때 어떤 문제가 예상되며, 다음 학습에서 무엇을 개선하면 좋을지 구체적으로 제안해 줘. 데이터 추가 수집의 우선순위도 함께.

AI 는 이런 질문에 굉장히 도움이 됩니다 — 숫자를 "무엇을 의미하는가"로 번역해 주고, 다음 액션을 제안합니다. 단, AI 의 제안 중 "데이터를 더 모으라"는 가장 흔한 조언이고 보통 옳지만, "어떤 데이터를 더 모을지"에 대해서는 우리집 환경을 아는 우리가 결정해야 합니다.

AI 와 함께 설계하기

위 모델(best.onnx)을 라즈베리파이 4에 올렸더니 — 추론은 잘 되지만 FPS가 3 정도 나온다. 추적에는 적어도 7~8 FPS가 필요한데, 다음 옵션들 중 어느 조합이 가장 효과적일까: 1) ONNX → NCNN 재변환 2) INT8 양자화 추가 3) 입력 해상도 640 → 416 4) Pi 4 → Pi 5 업그레이드 5) Coral USB Accelerator 추가 각 옵션의 비용(시간/돈)과 예상 FPS 향상, 정확도 영향을 표로 정리해 줘. 그리고 우리 상황(가정용 RC카, 예산 한정)에서 추천 우선순위도.

Chapter 6 핵심 정리

전이학습 — 사전학습 모델의 후반 층만 우리 데이터에 맞게 조정한다.

학습은 GPU에서 (PC GPU 또는 Colab). 라즈베리파이는 추론 전용.

주요 평가지표: Precision(오탐), Recall(누락), mAP(종합).

과적합 = 학습 데이터를 외운 상태. val loss가 다시 오르면 신호.

ONNX 변환 + INT8 양자화로 라즈베리파이 추론 속도 3~4배 향상.

NCNN은 ARM CPU에 최적화되어 라즈베리파이에서 더 빠를 때가 많다.

학습 결과 해석은 AI와 함께. 단, "내 환경에 맞는" 결정은 사람이 한다.

Chapter 7. GPS — 차량의 위치를 알다

지금까지 우리는 라즈베리파이의 "눈"을 만들었습니다. 카메라로 보고, AI로 인식하고, 좌표로 표현했습니다. 그러나 한 가지 빠진 것이 있습니다 — "내가 지금 어디에 있는가"라는 정보입니다. 카메라는 "내 앞에 무엇이 있는가"를 알려주지만, 차량 자체의 위치는 모릅니다.

RC카가 거실을 돌아다닐 때 위치 정보가 왜 필요할까요? 세 가지 이유가 있습니다.

- ① 행동 로깅 — 고양이를 따라다닌 경로를 기록하면, "우리집 고양이는 주로 어디에 있는가"라는 행동 패턴이 보입니다.
- ② 영역 제한 — "지정 구역 밖으로 나가지 마라"는 안전 규칙을 줄 수 있습니다.
- ③ Vol.5의 씨앗 — 지도 기반 자율주행으로 발전하는 첫걸음입니다.

이번 장은 GPS 모듈을 라즈베리파이에 붙이고, 위치 데이터를 받아 활용 가능한 형태로 가공하는 일을 다룹니다. 그리고 마지막으로 — 야외에서만 작동하는 GPS의 한계를 어떻게 보완할지도 짚습니다.

7.1 GPS의 원리 — 위성 4개와 삼변측량

GPS(Global Positioning System)는 지구 궤도를 도는 30여 개의 위성이 보내는 신호를 받아 "내가 지구의 어디에 있는가"를 계산하는 시스템입니다. 미국 국방부가 군사용으로 개발했지만 1980년대부터 민간에 개방되었고, 이제는 모든 스마트폰이 매일 쓰는 인프라가 되었습니다.

원리는 단순한 거리 측정

각 위성은 "지금 시각은 XX이고, 내 위치는 (x, y, z)다"라는 신호를 끊임없이 보냅니다. 지상의 수신기는 그 신호가 도착한 시각과 위성이 보낸 시각의 차이로 위성과의 거리를 계산합니다.

거리 = 빛의 속도 × 시간차

빛의 속도 $\approx 3 \times 10^8$ m/s

위성에서 신호가 출발한 시각: 12:00:00.000000

지상에서 신호가 도착한 시각: 12:00:00.067000

시간차 = 0.067 초

거리 = $3 \times 10^8 \times 0.067 = 20,100,000 \text{ m} = \text{약 } 2 \text{ 만 km}$

위성 한 개와의 거리만 알면 — 그 위성을 중심으로 한 "구의 표면" 위 어딘가에 있다는 뜻입니다. 위성 두 개의 거리를 알면 두 구의 교집합(원). 세 개면 두 점, 네 개면 한 점으로 좁혀집니다. 이것이 "삼변측량(trilateration)"입니다 — 흔히 알려진 "삼각측량(triangulation)"과는 다른 방법입니다. 삼각측량은 각도를 측정하지만, 삼변측량은 거리만 씁니다.

왜 위성이 4 개 필요한가

이론적으로는 위성 3 개면 위치가 한 점으로 정해집니다. 그런데 왜 4 개가 필요할까요? 답은 — 수신기의 시계 오차 때문입니다.

"시간차"를 정확히 측정하려면 수신기의 시계도 위성의 시계만큼 정확해야 합니다. 위성에는 원자시계가 들어 있지만, 수신기(스마트폰, GPS 모듈)에는 수정 진동자 시계뿐입니다. 수신기 시계가 1 만분의 1 초만 어긋나도 거리가 30km 어긋납니다. 그래서 GPS 는 "수신기의 시계 오차도 미지수로 두고" 4 개의 위성을 써서 4 개의 미지수(x, y, z, t)를 한꺼번에 푸는 방식으로 동작합니다. 결과적으로 수신기는 자기 시계도 위성 시계와 동기화시킵니다 — GPS 가 사실 매우 정확한 시계 동기화 도구이기도 한 이유입니다.

임베디드 개발자에게 의미:

이 "시간 동기화 부산물" 때문에 통신 인프라(이동통신 기지국, 금융 거래 시스템, 데이터센터)는 정확한 시각을 얻기 위해 GPS 를 씁니다. 우리 RC 카에서도 GPS 시각은 모든 로그의 "공통 시계"가 됩니다 — 카메라 인식 시각, GPS 위치 시각, TC237 센서 시각을 하나의 타임라인에 놓을 수 있습니다.

정확도와 한계

민간용 GPS 의 일반적인 정확도는 약 2~5 미터입니다. 더 정확한 "DGPS"나 "RTK"는 cm 급 정확도를 내지만, 보정 기지국이 필요하고 가격도 한 자릿수 위입니다. RC 카에는 일반 GPS 면 충분합니다.

GPS 의 결정적 한계는 두 가지입니다.

- 실내·지하·터널에서는 신호가 안 잡힌다 — GPS 신호는 위성에서 약 2 만 km

떨어진 곳에서 출발하므로, 도착할 때 매우 약합니다. 건물 지붕 한 층을 뚫기도 어렵습니다.

- 도심 빌딩 사이에서는 다중경로(multipath)로 부정확해진다 — 신호가 빌딩에 반사되어 도착하면 거리 계산이 어긋납니다.

그래서 RC 카를 처음 테스트할 때는 — 실외 공원, 운동장, 마당 같은 "하늘이 트인 곳"을 골라야 합니다. 거실 GPS 테스트는 시간 낭비입니다.

7.2 NEO-6M/NEO-M8N 모듈 비교와 NMEA 프로토콜

우리는 GPS 신호를 직접 디코딩하지 않습니다. 위성 신호 디코딩은 매우 복잡한 디지털 신호 처리(DSP) 작업입니다. 대신 u-blox 사의 GPS 모듈을 사용합니다 — 모듈이 위성 신호를 처리해 "위도·경도·시각"을 텍스트로 뽑아 UART 로 보내줍니다. 우리는 그 텍스트를 읽기만 하면 됩니다.

NEO-6M

가장 흔하고 저렴한(5~10\$) GPS 모듈입니다.

- 위치 정확도 — 약 2.5m
- 콜드 스타트 — 처음 위치를 잡기까지 30 초~수 분
- 핫 스타트 — 1 초 이내
- 수신 위성 — GPS 만
- 업데이트 주기 — 1~5Hz

입문용으로 충분합니다. 다만 도심 환경이나 부분적으로 가려진 곳에서는 fix 를 잃기 쉽습니다.

NEO-M8N

좀 더 비싸지만(15~25\$) 다중 위성 시스템을 받는 상위 모듈입니다.

- 위치 정확도 — 약 2m
- 콜드 스타트 — 약 26 초
- 수신 위성 — GPS + GLONASS + Galileo + BeiDou
- 업데이트 주기 — 최대 10Hz
- 도심 환경에서 NEO-6M 보다 안정적

다중 위성 시스템을 받는다는 점이 큰 장점입니다 — GPS 위성만 잘 보이지 않는

위치에서도 GLONASS 위성이 보이면 위치를 잡습니다. RC 카가 빠르게 움직이는 시나리오라면 10Hz 업데이트도 의미 있습니다.

어떤 것을 살까

Vol.4 RC 카는 천천히 움직이고, 야외 환경에서 동작합니다. 두 모듈 모두 가능하지만, 책에서는 NEO-M8N 을 기준으로 작성합니다 — 도심 환경에서 안정성 차이가 크기 때문입니다. 다만 NEO-6M 으로도 코드는 거의 동일하게 동작합니다.

NMEA 0183 프로토콜

두 모듈 모두 "NMEA 0183"이라는 표준 텍스트 프로토콜로 데이터를 보냅니다.

NMEA 는 "National Marine Electronics Association"의 약자로 — 원래 해양 항법 장비용으로 1980 년대에 정해진 표준입니다. 지금도 거의 모든 GPS 모듈이 이 형식을 씁니다.

NMEA 메시지는 ASCII 텍스트 한 줄입니다. 예를 들면 다음과 같습니다.

```
$GPGGA,165725.00,3736.6700,N,12700.5000,E,1,08,1.0,50.0,M,18.0,M,,*5C
$GPRMC,165725.00,A,3736.6700,N,12700.5000,E,0.5,054.7,260524,,*0A
$GPGSV,3,1,12,02,68,304,32,05,15,224,21,06,29,278,28,12,52,061,31*7E
```

각 줄은 \$ 로 시작하고 *XX 체크섬으로 끝난다.

중간은 콤마로 구분된 필드.

우리에게 가장 중요한 메시지 두 개는 GGA 와 RMC 입니다.

GPGGA — 위치 fix 데이터

```
$GPGGA,165725.00,3736.6700,N,12700.5000,E,1,08,1.0,50.0,M,18.0,M,,*5C
```

필드 0: \$GPGGA

필드 1: 165725.00 — UTC 시각 (16 시 57 분 25 초)

필드 2: 3736.6700 — 위도 (37 도 36.6700 분)

필드 3: N — 북위(N) / 남위(S)

필드 4: 12700.5000 — 경도 (127 도 00.5000 분)

필드 5: E — 동경(E) / 서경(W)

필드 6: 1 — fix 상태 (0=없음, 1=GPS, 2=DGPS)

필드 7: 08 — 수신 위성 수

필드 8: 1.0 — HDOP (수평 정밀도 계수)
필드 9: 50.0 — 고도(m)
... (이후 생략)

주의할 점은 위도/경도 표기가 "도분(DDMM.MMMM)" 형식이라는 것입니다. 일반적인 "도(DD.DDDD)" 형식이 아닙니다. 3736.6700 은 "37 도 36.67 분"이고, 십진수 도로 환산하면 $37 + 36.67/60 = 37.6112$ 도입니다. 이 변환은 자주 실수하는 부분이니 라이브러리를 쓰는 것이 안전합니다.

GPRMC — 위치 + 속도 + 방위

```
$GPRMC,165725.00,A,3736.6700,N,12700.5000,E,0.5,054.7,260524,,*0A
```

필드 0: \$GPRMC
필드 1: 165725.00 — UTC 시각
필드 2: A — 상태 (A=유효, V=무효)
필드 3-6: 위도/경도 (GGA 와 동일)
필드 7: 0.5 — 속도 (노트, 1 노트 $\approx$ 0.514 m/s)
필드 8: 054.7 — 방위 (도, 진북 기준)
필드 9: 260524 — 날짜 (26 일 5 월 24 년)

이 두 메시지만 잘 파싱하면 RC 카가 필요한 정보(시각·위도·경도·속도·방위)를 다 얻을 수 있습니다.

7.3 라즈베리파이 UART 로 GPS 데이터 파싱

NEO 모듈을 라즈베리파이에 연결합니다. UART 통신이라 결선은 단순합니다.

NEO-M8N → Raspberry Pi

VCC → Pin 2 (5V) 또는 Pin 1 (3.3V, 모듈 따라 다름)
GND → Pin 6 (GND)
TX → Pin 10 (GPIO15, UART RX)
RX → Pin 8 (GPIO14, UART TX) — 사용 안 할 수도 있음

주의: GPS 의 TX 가 라즈베리파이의 RX 로 연결됨 (크로스).

대부분의 NEO-M8N 보드는 5V 입력을 받아도 동작하지만, 데이터 핀은 3.3V

레벨입니다. 라즈베리파이의 GPIO 도 3.3V 레벨이라 — 직접 연결해도 문제없습니다. (Arduino 의 5V GPIO 에 연결할 때는 레벨 시프터가 필요한 경우가 있습니다.)

Pi UART 활성화

Raspberry Pi 의 기본 UART(GPIO14/15)는 Bluetooth 가 점유하고 있어서, GPS 용으로 쓰려면 Bluetooth 를 비활성화해야 합니다. /boot/firmware/config.txt 에 다음 줄을 추가하고 재부팅합니다.

```
sudo nano /boot/firmware/config.txt
# (Bookworm 이전 OS 는 /boot/config.txt)
```

다음 두 줄 추가:

```
enable_uart=1
dtoverlay=disable-bt
```

저장 후:

```
sudo systemctl disable hciuart
sudo reboot
```

재부팅 후 /dev/serial0 또는 /dev/ttyAMA0 가 GPS 모듈에 연결됩니다. 둘 다 같은 장치를 가리키는 별명입니다 — /dev/serial0 가 더 안정적인 별명이므로 권장합니다.

최소 파싱 코드

pyserial 로 직렬 포트를 열고, pynmea2 로 NMEA 메시지를 파싱합니다.

```
import serial
import pynmea2

ser = serial.Serial("/dev/serial0", 9600, timeout=1)

while True:
    line = ser.readline().decode("ascii", errors="ignore").strip()
    if not line.startswith("$"):
        continue
    try:
        msg = pynmea2.parse(line)
```

```

if msg.sentence_type == "GGA" and msg.gps_qual > 0:
    print(f"위도 {msg.latitude:.6f}, 경도 {msg.longitude:.6f}, "
          f"위성 {msg.num_sats}개, 고도 {msg.altitude}m")
except pynmea2.ParseError:
    continue

```

이 코드만으로 GPS 위치가 콘솔에 1 초마다 찍힙니다. `msg.latitude` 와 `msg.longitude` 는 이미 십진수 도(degree)로 변환되어 나옵니다 — `pynmea2` 가 NMEA 의 도분 형식을 십진수로 자동 변환해 줍니다.

Fix 를 못 잡을 때

야외로 들고 나갔는데도 fix 가 안 잡힌다면 다음을 점검합니다.

- LED 확인 — NEO 모듈의 PPS LED 가 깜빡이면 fix 성공. 안 깜빡이면 위성을 못 잡는 중.
- 안테나 확인 — 안테나가 평평한 면이 하늘을 향해야 합니다. 뒤집혀 있거나 손가락으로 가려져 있으면 신호가 약합니다.
- 위치 확인 — 빌딩 사이, 큰 나무 아래, 차량 안에서는 fix 가 어렵습니다. 가능한 트인 공간으로.
- 시간 확인 — 콜드 스타트는 30 초~수 분이 걸립니다. "안 된다"고 판단하기 전에 5 분은 기다리세요.
- 배선 확인 — TX/RX 가 바뀌어 연결되면 데이터가 안 옵니다. "\$"로 시작하는 줄이 하나도 안 보이면 의심하세요.

7.4 위도·경도로 두 지점 간 거리 계산 (하버사인 공식)

GPS 로 얻은 위도·경도 두 쌍이 있다면, 두 점 사이의 실제 거리는 어떻게 계산할까요? 지구가 평면이라면 피타고라스를 쓰면 됩니다. 그러나 지구는 구형이라 — 큰 거리에서는 곡률을 고려해야 합니다.

이때 쓰는 것이 "하버사인 공식(Haversine formula)"입니다.

```

import math

def haversine(lat1, lon1, lat2, lon2):
    """두 지점 사이 거리를 미터 단위로 반환."""

```

```

R = 6371000 # 지구 평균 반지름 (m)
phi1 = math.radians(lat1)
phi2 = math.radians(lat2)
d_phi = math.radians(lat2 - lat1)
d_lam = math.radians(lon2 - lon1)

a = (math.sin(d_phi / 2) ** 2 +
      math.cos(phi1) * math.cos(phi2) * math.sin(d_lam / 2) ** 2)
c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
return R * c

# 사용 예: 서울 시청 → 강남역
d = haversine(37.5665, 126.9780, 37.4979, 127.0276)
print(f"{d:.0f} m") # 약 8800m

```

우리 RC 카의 사용 범위는 길어야 수십~수백 미터입니다. 이런 짧은 거리에서는 평면 근사(피타고라스)도 거의 같은 답을 줍니다. 하지만 일반적인 도구로 하버사인을 알아두면 — 나중에 "우리집에서 공원까지 RC 카가 얼마나 멀리 갔는가" 같은 계산에 그대로 쓸 수 있습니다.

실용 함수 — "내가 시작점에서 얼마나 멀어졌나"

RC 카에서 자주 쓸 함수입니다.

```

class TripTracker:
    def __init__(self):
        self.start = None
        self.total_distance = 0.0
        self.last_pos = None

    def update(self, lat, lon):
        if self.start is None:
            self.start = (lat, lon)
            self.last_pos = (lat, lon)
            return

        # 누적 이동 거리
        step = haversine(self.last_pos[0], self.last_pos[1], lat, lon)
        # GPS 노이즈로 1m 이하 미동은 무시 (정지 중일 때 누적되는 거 방지)
        if step > 1.0:

```

```

        self.total_distance += step
        self.last_pos = (lat, lon)

    def distance_from_start(self):
        if self.start is None or self.last_pos is None:
            return 0.0
        return haversine(*self.start, *self.last_pos)

```

이 클래스를 통합 시스템(8 장)에서 쓰면 — "RC 카가 출발점에서 50m 이상 멀어지면 정지" 같은 영역 제한이 간단하게 구현됩니다.

7.5 GPS 데이터 로깅과 시각화

RC 카가 어디를 돌아다녔는지를 기록하면, 나중에 "고양이가 어디로 갔는가"를 분석할 수 있습니다. 가장 간단하면서도 실용적인 방법을 살펴봅시다.

CSV 로깅

순수 텍스트로 저장합니다. 나중에 Excel·Python·Google Sheets 등 어디서든 열어볼 수 있습니다.

```

import csv
from datetime import datetime, timezone

with open("track.csv", "a", newline="") as f:
    writer = csv.writer(f)
    # 헤더 한 번만
    if f.tell() == 0:
        writer.writerow(["timestamp", "lat", "lon", "speed_mps", "satellites"])

    # ... GPS 루프 안에서:
    timestamp = datetime.now(timezone.utc).isoformat()
    speed_mps = msg.spd_over_grnd * 0.514 # 노트 → m/s
    writer.writerow([timestamp, msg.latitude, msg.longitude, speed_mps,
msg.num_sats])
    f.flush() # 강제 디스크 쓰기 (전원 끊겨도 손실 최소화)

```

flush()를 꼭 호출하세요 — RC 카는 전원이 갑자기 끊길 수 있는 환경입니다. flush 없으면 마지막 수집 줄이 버퍼에만 남아 사라질 수 있습니다.

KML 로 변환 — Google Earth 에서 보기

CSV 는 숫자 더미일 뿐이라 "어떤 길로 갔는지"를 한눈에 보기 어렵습니다. KML 로 변환하면 Google Earth 나 Google Maps 에서 경로가 선으로 그려집니다.

```
import csv

def csv_to_kml(csv_path, kml_path):
    coords = []
    with open(csv_path) as f:
        reader = csv.DictReader(f)
        for row in reader:
            coords.append(f"{row['lon']},{row['lat']},0")

    kml = f"""<?xml version="1.0"?>
<kml xmlns="http://www.opengis.net/kml/2.2">
  <Document>
    <Placemark>
      <name>RC car track</name>
      <LineString>
        <coordinates>
          {' '.join(coords)}
        </coordinates>
      </LineString>
    </Placemark>
  </Document>
</kml>"""
    with open(kml_path, "w") as f:
        f.write(kml)

csv_to_kml("track.csv", "track.kml")
```

KML 파일을 Google Earth 에 드래그&드롭하면 — 지도 위에 RC 카의 이동 경로가 빨간 선으로 그려집니다. 학습 데이터 수집할 때 RC 카가 어느 영역을 커버했는지를 시각적으로 점검할 수 있어 매우 유용합니다.

Folium 으로 인터랙티브 지도

Python 에서 바로 인터랙티브 지도를 만들고 싶다면 folium 라이브러리가 편합니다.

```
import folium
```

```
import csv

points = []
with open("track.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        points.append([float(row['lat']), float(row['lon'])])

m = folium.Map(location=points[0], zoom_start=18)
folium.PolyLine(points, color="red", weight=3).add_to(m)
folium.Marker(points[0], popup="Start").add_to(m)
folium.Marker(points[-1], popup="End").add_to(m)
m.save("track.html")
```

결과 HTML 파일을 브라우저에서 열면 — 지도 위에서 줌인/줌아웃하며 경로를 살펴볼 수 있습니다.

7.6 GPS의 한계 — 그리고 실내에서는?

앞서 짚었듯 GPS는 실외용입니다. 그런데 우리집 고양이는 보통 실내에서 활동하지요. 모순처럼 보이는 이 상황을 어떻게 풀까요?

선택 1 — 실외만 다룬다

RC 카를 마당이나 공원에서 운영합니다. GPS가 잘 잡히고, 카메라도 자연광에서 더 잘 동작합니다. 단, 우리집 고양이를 야외로 데리고 나가는 것이 가능한 환경이어야 합니다.

선택 2 — 실내에서는 GPS를 비활성

실내 동작 시에는 GPS 로깅을 끄고, 카메라 + IMU 기반 추적만 동작합니다. 7장의 GPS는 "있으면 더 좋은 기능"으로 두고, 추적 자체에는 필수가 아닙니다.

선택 3 — 실내 측위로 보완

실내에서도 위치를 알고 싶다면 다른 기술을 추가합니다.

- Wi-Fi RSSI 기반 — 주변 Wi-Fi 신호 세기로 위치 추정. 정확도 3~5m.
- BLE 비콘 — 집안에 여러 개 두고 신호 세기 측정. 정확도 1~3m.
- UWB(Ultra-Wide Band) — Apple AirTag에 쓰이는 기술. cm급 정확도지만

가격 비쌘.

- Visual SLAM — 카메라로 주변 환경을 보고 자기 위치를 추정. 무겁지만 정확.

이 책에서는 GPS 만 다룹니다. 실내 측위는 그 자체로 책 한 권 분량의 주제이고, RC 카의 첫 버전에는 과합니다.

실용적 권장:

처음 만들 때는 — 카메라 + AI + 모터 제어를 마당이나 공원에서 충분히 테스트하세요. GPS 로깅은 그 후에 추가합니다. 실내 측위는 "필요해질 때" 고민하면 됩니다. 시스템을 한꺼번에 완벽하게 만들려 하지 말고, 단계적으로 늘려가는 것이 임베디드 프로젝트의 정답입니다.

7.7 AI 와 함께하는 GPS 로깅 실습

이번 실습은 "이동 경로를 실시간 KML 로 저장하는 백그라운드 데몬"입니다. AI 에게 통합된 해법을 부탁드립니다.

AI 와 함께 설계하기

라즈베리파이 + NEO-M8N 환경에서 다음을 동작하는 백그라운드 데몬 스크립트를 작성해 줘: 1) /dev/serial0 에서 NMEA 데이터 수신 (9600 baud) 2) GPS fix 가 잡혔을 때만 (lat, lon, speed_mps, satellites)를 5 초마다 CSV 에 추가 3) 동시에 실시간 KML 파일도 갱신 (Google Earth 에서 바로 보기) 4) 30 초 이상 fix 가 끊기면 syslog 에 경고 기록 5) Ctrl+C 또는 SIGTERM 으로 종료 시 파일을 안전하게 닫기 (flush + close) 6) 로그 파일이 10MB 넘으면 자동 회전 (오래된 것 보관) 코드와 함께 systemd 서비스로 등록하는 .service 파일도 보여줘. 부팅 시 자동 시작되게.

AI 가 만든 스크립트를 받았다면, 그대로 실행하기 전에 다음을 점검합니다.

- SIGTERM 처리가 안전한가 — 갑작스러운 종료에도 파일이 깨지지 않아야 합니다.
- 예외 처리가 충분한가 — 직렬 포트가 끊기거나 디스크가 가득 차면? 데몬이 죽지 않고 회복해야 합니다.
- 리소스 누수 없는가 — 장시간 동작 시 메모리·파일 핸들이 새지 않는지 확인.

AI 와 함께 설계하기

위 GPS 데몬 코드를 24 시간 연속 동작시켰을 때 메모리 누수·파일 핸들 누수가 없는지

검증하는 방법을 알려줘. 라즈베리파이에서 사용할 수 있는 도구(/proc, ps, htop, valgrind 등) 위주로 — 우리집 환경에서 실용적인 것만.

Chapter 7 핵심 정리

GPS 는 위성 4 개와의 거리로 삼변측량 — 한 개는 수신기 시계 오차 보정용.

GPS 는 실외용. 실내·지하·터널에서는 신호가 거의 안 잡힌다.

NEO-6M 은 입문, NEO-M8N 은 다중 위성(GPS+GLONASS+Galileo)으로 더 안정적.

Pi UART(GPIO14/15)는 기본적으로 Bluetooth 가 점유. disable-bt 후 /dev/serial0 사용.

NMEA 메시지는 ASCII 텍스트 — pynmea2 가 위경도 변환까지 자동 처리.

두 지점 거리는 하버사인 공식. 짧은 거리는 평면 근사도 가능.

로깅에는 CSV + 시각화에는 KML/Folium. flush()로 전원 손실 대비.

GPS 는 "있으면 좋은" 기능. 추적 자체는 카메라 + AI 만으로도 가능.

Chapter 8. 통합 — 고양이를 따라가는 RC 카 (Physical AI 완성)

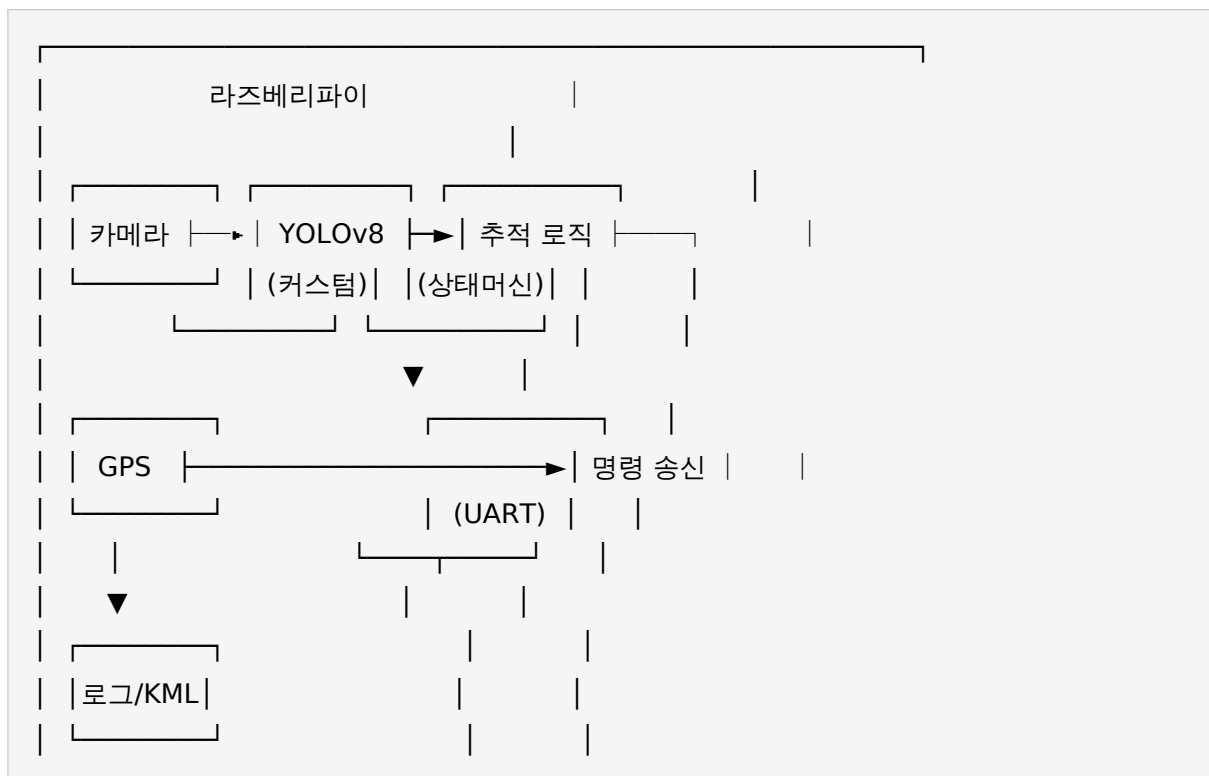
드디어 마지막 챕터입니다. 1장에서 7장까지 — 우리는 부품 하나하나, 기능 하나하나를 따로 만들어 왔습니다. 라즈베리파이를 셋업했고, 카메라로 영상을 받았고, YOLO로 고양이를 인식했고, 우리집 고양이를 학습시켰고, GPS로 위치를 받았습니다. 이제 이 모든 조각을 하나로 묶어 — RC 카 한 대가 자율적으로 고양이를 따라가도록 만듭니다.

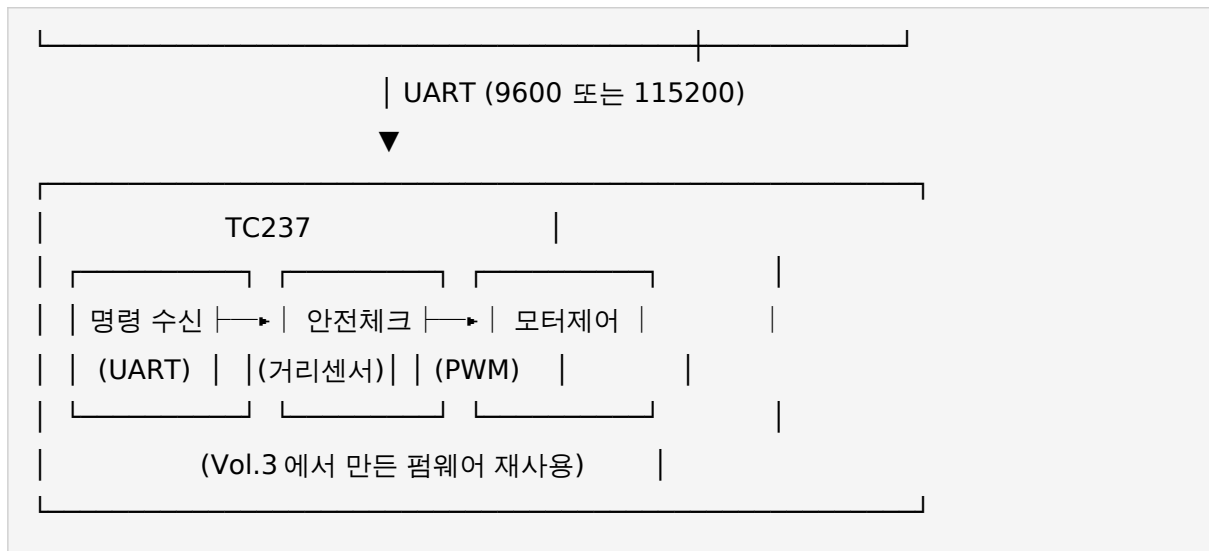
그러나 이 통합은 단순한 "코드 합치기"가 아닙니다. 시스템 통합은 그 자체로 별개의 기술입니다. 좋은 부품들을 모아도 통합이 엉성하면 — 전체 시스템은 작동하지 않습니다. 자동차 업계에서 "시스템 엔지니어"라는 직종이 따로 있는 이유입니다.

이번 장은 통합의 핵심 기술 — 아키텍처 설계, 상태 관리, 양 보드 간 통신 프로토콜, 안전 우선 원칙 — 을 차례로 다룹니다. 그리고 마지막에 모든 것을 묶은 통합 코드와 실제 운영 시 점검 사항으로 마무리합니다.

8.1 추적 시스템 전체 아키텍처

드디어 모든 조각이 모이는 자리입니다. 1~7장에서 각각 따로 동작했던 부품들을 한 시스템으로 묶습니다. 시스템 구성도부터 봅시다.





핵심 설계 원칙 — 책임의 분리

이 아키텍처의 가장 중요한 원칙은 "책임의 분리"입니다. 라즈베리파이와 TC237 이 각자 다른 책임을 집니다.

- 라즈베리파이 = 인지(Perception) + 의사결정(Decision) — "무엇을 할지"를 정한다.
- TC237 = 실행(Execution) + 안전(Safety) — "안전하게 어떻게 할지"를 정한다.

이 원칙의 결정적 효과 — 라즈베리파이가 잘못 판단해도(예: AI 가 벽을 고양이로 오인식), TC237 이 거리 센서로 충돌을 막습니다. AI 는 추론이 빠르지만 비결정적이고, MCU 는 느리지만 결정론적입니다. 두 특성을 결합해 — "AI 의 지능 + MCU 의 안전성"이라는 새로운 시스템을 만듭니다.

실제 자동차와의 평행:

이 구조는 1.3 절에서 다룬 ADAS 구조와 정확히 같은 패턴입니다. Tesla 의 FSD 컴퓨터, Mobileye 의 EyeQ 같은 비전 SoC 가 "무엇을 할지"를 결정하고, AURIX 같은 안전 MCU 가 "안전하게 어떻게 할지"를 결정합니다. 우리의 RC 카는 — 같은 설계 철학의 작은 시제품입니다.

프로세스 구조

라즈베리파이 안에서도 작업을 어떻게 나눌지 결정해야 합니다. 한 프로세스에 모든 코드를 넣어도 동작은 하지만, 추적이 끊기거나 GPS 가 막히는 등 안정성 문제가

생기기 쉽습니다.

권장하는 구조는 다음과 같습니다.

방식 1: 단일 프로세스, 멀티스레드 (간단)

- 메인 스레드: 카메라 캡처 + YOLO 추론 + 명령 송신
 - GPS 스레드: NMEA 파싱 + 위치 갱신
- 공유 변수에 lock 으로 동시 접근 제어.

방식 2: 멀티 프로세스 (안정적)

- 추적 프로세스: 카메라 + YOLO + 명령 (실시간성 중요)
 - GPS 프로세스: NMEA + 로깅 (별도 systemd 데몬)
- 프로세스 간 통신은 파일 또는 UNIX 소켓.

권장: 학습 단계에서는 방식 1, 실전 운영은 방식 2.

이번 챕터의 통합 코드는 방식 1로 작성합니다. 한 파일에서 시작 → 동작 확인 → 점진적으로 분리, 가 자연스러운 흐름입니다.

8.2 객체의 화면 좌표를 조향 의사결정으로 변환하기

카메라 영상에서 고양이의 바운딩 박스를 얻었다고 합시다. 이걸 "왼쪽으로 돌아라" / "앞으로 가라" 같은 명령으로 어떻게 바꿀까요? 의사결정 로직의 핵심입니다.

단순한 규칙 — 화면 영역 분할

가장 단순한 방법은 화면을 좌·중·우 세 영역으로 나누고 — 고양이의 중심이 어느 영역에 있는지에 따라 조향하는 것입니다.

```
# 화면 가로 폭 640 기준
W = 640
cx = (x1 + x2) / 2    # 고양이 박스 중심 x

if cx < W * 0.4:
    cmd = "TURN_LEFT"
elif cx > W * 0.6:
    cmd = "TURN_RIGHT"
else:
    cmd = "FORWARD"
```

이게 동작은 합니다. 단, 행동이 거칩니다 — 고양이가 화면 한가운데 있다가 살짝 왼쪽으로 가면 갑자기 "LEFT" 명령이 떨어집니다. 시스템이 떨립니다(chattering). Vol.3 의 자이로 보정에서 이미 본 문제와 같은 패턴입니다 — 임계값으로 끊는 디지털 제어의 본질적 한계.

부드러운 제어 — 비례 제어(P 제어)

더 부드럽게 하려면 "화면 중심에서 얼마나 벗어났는가"를 비례적으로 조향 강도로 변환합니다. 이것이 P 제어(Proportional Control)입니다.

```
offset = (cx - W / 2) / (W / 2) # -1.0 ~ +1.0
# 음수면 왼쪽, 양수면 오른쪽

steering_strength = int(offset * 100) # -100 ~ +100
cmd = f"STEER {steering_strength}"

# 예:
# cx = 320 (정중앙) → offset 0 → STEER 0 (직진)
# cx = 240 → offset -0.25 → STEER -25 (약하게 왼쪽)
# cx = 100 → offset -0.69 → STEER -69 (강하게 왼쪽)
```

이렇게 하면 고양이가 살짝만 벗어나도 살짝만 돌고, 많이 벗어나면 많이 돌게 됩니다. 자연스러운 추적이 됩니다.

실제 자동차 비유:

이 비례 제어는 차선 유지(LKA, Lane Keep Assist)에서 쓰는 방식과 본질적으로 같습니다. 차선 중심에서 벗어난 거리를 측정해, 그 비율만큼 핸들을 돌립니다. 우리는 차선 대신 고양이를 중심으로 삼는 것뿐입니다. 산업용 트럭의 자동 도킹, 드론의 객체 추적, 농업 로봇의 줄 따라가기 — 같은 비례 제어 원리가 어디서나 반복됩니다.

진보된 제어 — PD 제어

P 제어로도 어느 정도 추적이 되지만, 한 가지 문제가 남습니다. 고양이가 빠르게 옆으로 움직이면 — RC 카가 "늦게" 따라갑니다. 항상 "고양이가 있던 자리"로 향하기 때문입니다.

이를 보완하려면 "고양이가 어느 방향으로 얼마나 빠르게 움직이고 있는가"를 같이

고려합니다. 즉 변화율(미분)을 더하는 것 — PD 제어입니다.

```
prev_offset = 0
def steering_pd(cx, dt=0.1):
    global prev_offset
    offset = (cx - W / 2) / (W / 2)
    derivative = (offset - prev_offset) / dt
    prev_offset = offset

    Kp = 100 # P 게인
    Kd = 30 # D 게인
    return int(Kp * offset + Kd * derivative)
```

Kp 는 "현재 오차에 얼마나 반응", Kd 는 "오차 변화에 얼마나 반응"입니다. Kd 가 너무 크면 떨림이 심해지고, 너무 작으면 효과가 미미합니다. 실험으로 찾습니다 — 우리 시스템에서는 Kp=100, Kd=30 정도가 좋은 출발점입니다.

정통 제어 이론에서는 여기에 "누적 오차(I 제어)"를 더한 PID 제어가 표준이지만 — RC 카의 시각 추적에는 P 또는 PD 로 충분한 경우가 많습니다. 누적 오차가 큰 의미가 없기 때문입니다.

8.3 바운딩 박스 크기로 거리 추정하기

좌우 조향은 했지만, "고양이까지 얼마나 가까운가"를 모르면 — RC 카가 멈출 줄을 모르고 들이박을 수 있습니다. 거리 추정이 필요합니다.

단일 카메라로 거리를 알 수 있는가?

이론적으로 단일 카메라(monocular)로는 절대적 거리를 알 수 없습니다. 같은 사이즈의 박스가 큰 물체가 멀리 있을 때와, 작은 물체가 가까이 있을 때 똑같이 보일 수 있기 때문입니다.

그러나 우리는 결정적 단서가 있습니다 — "우리집 고양이의 실제 크기를 안다"는 것입니다. 모르는 물체라면 모호하지만, "높이 25cm 정도의 고양이"라는 사전 지식이 있으면 — 박스 크기에서 거리를 역산할 수 있습니다.

핀홀 카메라 모델

카메라의 기하학을 매우 단순화한 "핀홀 카메라 모델"이 출발점입니다. 다음

관계식이 성립합니다.

$$\text{박스_높이_픽셀} = (\text{초점거리} \times \text{실제_높이}) / \text{거리}$$

$$\rightarrow \text{거리} = (\text{초점거리} \times \text{실제_높이}) / \text{박스_높이_픽셀}$$

여기서:

초점거리(f) — 카메라 내부 파라미터, 픽셀 단위

실제_높이 — 고양이의 실제 높이, 미터 단위 ($\approx 0.25\text{m}$)

박스_높이_픽셀 — YOLO가 뽑아준 박스 높이

거리 — 카메라부터 고양이까지의 거리, 미터 단위

간단한 캘리브레이션

초점거리를 데이터시트에서 찾는 대신, 실측으로 캘리브레이션합니다.

캘리브레이션 절차:

1. 고양이(또는 비슷한 크기의 인형)를 정확히 1m 거리에 놓습니다.
2. 카메라로 사진을 찍고 YOLO를 돌립니다.
3. 박스 높이 픽셀을 기록합니다.

예: 1m에서 박스 높이 = 240 픽셀.

이 한 쌍이 있으면 이후 거리 추정 가능:

$$\text{거리} = 1.0\text{m} \times (240 / \text{박스_높이})$$

따라서:

박스 높이 120 → 거리 약 2m

박스 높이 80 → 거리 약 3m

박스 높이 480 → 거리 약 0.5m

실용적 거리 추정 함수

```
def estimate_distance(box_height_px,
                      ref_height_px=240,
                      ref_dist_m=1.0):
    """단일 카메라 거리 추정 (정면 자세 가정)."""
    if box_height_px < 20:
```



```

        return None # 너무 작아서 신뢰할 수 없음
    return ref_dist_m * ref_height_px / box_height_px

# 사용:
dist = estimate_distance(y2 - y1)
if dist is None:
    cmd = "SEARCH" # 너무 멀거나 부분적으로 보임
elif dist < 0.5:
    cmd = "STOP" # 너무 가까움
elif dist < 1.0:
    cmd = "FORWARD_SLOW"
else:
    cmd = "FORWARD_FAST"

```

이 방법의 한계

이 단일 카메라 거리 추정에는 "고양이가 정면을 보고 있을 때"를 가정합니다. 옆모습이거나 누워 있으면 박스 비율이 달라져 거리 추정이 어긋납니다. 또한 — 고양이마다 실제 크기가 다르므로 (어린 고양이 vs 어른 고양이), 캘리브레이션 시 "내 고양이"로 맞춰야 합니다.

정확한 거리가 필요하다면 다른 센서를 추가합니다.

- 스테레오 카메라 — 두 카메라의 시차로 깊이 계산. 라즈베리파이 호환 모듈이 있음.
- ToF(Time-of-Flight) 센서 — VL53L1X 같은 초소형 거리 센서. cm 급 정확도.
- 초음파 — Vol.3 에서 다룬 HC-SR04. TC237 측에서 이미 동작 중이므로 이 데이터를 라즈베리파이로 받아오는 것이 가장 간단함.

실용적 권장은 — TC237 에서 초음파 데이터를 UART 로 회신해 라즈베리파이가 받는 구조입니다. 어차피 안전을 위해 TC237 이 거리를 측정하고 있으니, 그 정보를 활용하면 됩니다. 이미 만든 시스템을 재활용하는 셈입니다.

8.4 추적 상태 머신 — 탐색·추적·농침·복귀

실제 추적은 "고양이가 보일 때만 따라가는" 단순한 일이 아닙니다. 고양이가 잠시 사라졌다가 다시 나타나는 일이 흔합니다. 가구 뒤로 갔다가, 모서리를 돌았다가, 카메라 사각지대에 들어갔다가 나오기를 반복합니다.

이런 동적인 상황을 깔끔하게 다루려면 — 상태 머신(State Machine)이 필요합니다. Vol.3 의 모터 제어 상태 관리에서도 같은 패턴을 썼습니다.

상태 정의

상태:

SEARCH — 고양이가 안 보임. 천천히 회전하며 찾는다.

TRACKING — 고양이가 보임. 조향하며 따라간다.

LOST — 직전에 보였는데 지금은 안 보임.

잠시 직진하며 기다린다 (잠시 가려진 상황 가정).

STOPPED — 너무 가까움 또는 장애물 감지. 정지.

IDLE — 사용자가 명시적으로 멈춤. 외부 명령 대기.

전이 규칙:

IDLE + 시작 명령 → SEARCH

SEARCH + 고양이 발견 → TRACKING

SEARCH + 30 초 경과 → IDLE (포기)

TRACKING + 고양이 사라짐(<2 초) → LOST

TRACKING + 거리 < 50cm → STOPPED

TRACKING + TC237 장애물 감지 → STOPPED

LOST + 고양이 재발견 → TRACKING

LOST + 2 초 경과 → SEARCH

STOPPED + 고양이 멀어짐 또는 사라짐 → LOST 또는 TRACKING

(어디서나) + 사용자 정지 명령 → IDLE

Python 구현 — 단순한 상태 머신

```
import time

class TrackingFSM:
    def __init__(self):
        self.state = "IDLE"
        self.last_seen = 0
        self.search_start = 0

    def update(self, cat_detected, distance_m, obstacle_near):
        now = time.time()
```

```
# 우선순위: 장애물 감지는 어떤 상태에서도 STOPPED
```

```
if obstacle_near:  
    return self._goto("STOPPED")
```

```
if self.state == "IDLE":  
    return "STOP" # 외부 트리거 대기
```

```
if self.state == "SEARCH":  
    if cat_detected:  
        self._goto("TRACKING")  
        self.last_seen = now  
        return "STEER 0" # 바로 정면을 향하도록  
    elif now - self.search_start > 30:  
        return self._goto("IDLE")  
    else:  
        return "SEARCH" # 천천히 회전
```

```
if self.state == "TRACKING":  
    if not cat_detected:  
        self._goto("LOST")  
        return "FORWARD_SLOW"  
    if distance_m and distance_m < 0.5:  
        return self._goto("STOPPED")  
    self.last_seen = now  
    return None # 추적은 외부에서 PD 로 결정
```

```
if self.state == "LOST":  
    if cat_detected:  
        self._goto("TRACKING")  
        self.last_seen = now  
        return None  
    if now - self.last_seen > 2:  
        self._goto("SEARCH")  
        self.search_start = now  
        return "SEARCH"  
    return "FORWARD_SLOW"
```

```
if self.state == "STOPPED":  
    if not obstacle_near and cat_detected:
```

```
        self._goto("TRACKING")
        return None
    return "STOP"
```

```
def _goto(self, new_state):
    self.state = new_state
    return None
```

상태 머신은 "무엇을 할지"를 정합니다. "어떻게 조향할지"는 별도 — 8.2의 PD 제어가 담당합니다. 두 모듈의 책임이 분리되어 있어, 각각 따로 디버깅·튜닝할 수 있습니다.

8.5 GPS 추적 로그 — 고양이의 이동 경로 기록하기

RC카가 고양이를 따라가는 동안 GPS가 동작하면, 자동으로 "고양이가 다닌 길"이 기록됩니다. 7장의 GPS 로깅 코드를 추적 시스템과 같은 프로세스에 통합합니다.

이 데이터는 단순한 재미가 아닙니다 — "우리집 고양이는 주로 어느 시간대에 어디로 다니는가"라는 행동 분석 데이터가 됩니다.

- 아침에는 거실 → 부엌 동선
- 오후에는 햇볕이 드는 창가에서 오래 머무름
- 저녁에는 침실로 이동

이런 패턴이 며칠치 누적되면 — 행동 패턴 분석 데이터가 됩니다. 실제로 야생동물 추적에 쓰이는 GPS 콜라(collar)의 원리와 같습니다. 우리는 야생 호랑이 대신 거실 고양이를 추적할 뿐입니다.

추적 + 위치 동기화

매 추적 사이클마다 "현재 추정 위치"를 같이 기록하면, 나중에 "이 위치에서 고양이를 잃었다" 같은 분석이 가능합니다.

```
import csv, time

with open("track.csv", "a", newline="") as f:
    writer = csv.writer(f)
    if f.tell() == 0:
        writer.writerow([
```

```

        "timestamp", "state", "cat_detected",
        "cat_cx", "cat_cy", "est_distance_m",
        "lat", "lon", "command"
    ])
# 메인 루프 안에서:
writer.writerow([
    time.time(), fsm.state, detected,
    cx if detected else None,
    cy if detected else None,
    dist,
    gps.latitude, gps.longitude,
    last_command
])
if i % 30 == 0:
    f.flush() # 30 프레임마다 디스크 쓰기

```

8.6 라즈베리파이와 TC237의 UART 통신 프로토콜

두 보드 사이의 약속을 정합니다. Vol.3에서 만든 명령 프로토콜과 호환되도록 합니다.

프로토콜 설계 원칙

좋은 시리얼 프로토콜은 다음 특성을 가집니다.

- 프레임 — 시작/끝 마커로 메시지 경계가 분명.
- 체크섬 — 비트 오류 검출.
- 응답 — 송신 측이 "명령이 도달했는가"를 확인할 수 있어야 함.
- 타임아웃 — 응답이 없을 때 시스템이 멈추지 않도록.
- 재시도 — 일시적 오류에서 복구할 수 있도록.

명령 포맷 (라즈베리파이 → TC237)

명령 패킷 구조 (6 바이트):

```

<STX> <CMD> <PARAM1> <PARAM2> <CHECKSUM> <ETX>
1B  1B  1B   1B   1B   1B

```

STX = 0x02, ETX = 0x03

CMD:

0x01 FORWARD	PARAM1: 속도(0~100)	PARAM2: 무시
0x02 STOP	PARAM1: 무시	PARAM2: 무시
0x03 STEER	PARAM1: 강도(0~100)	PARAM2: 0=좌, 1=우
0x04 SEARCH	PARAM1: 회전속도(0~100)	PARAM2: 0=좌, 1=우
0x05 STATUS_REQ	PARAM1: 무시	PARAM2: 무시
0x06 EMERGENCY	PARAM1: 무시	PARAM2: 무시 (즉시 정지)

CHECKSUM = CMD XOR PARAM1 XOR PARAM2

응답 포맷 (TC237 → 라즈베리파이)

TC237 → 라즈베리파이 방향은 JSON 한 줄(텍스트)을 씁니다. JSON 은 사람이 읽기 쉽고, 라즈베리파이에서 파싱이 간단합니다. TC237 측은 `sprintf` 같은 함수로 직접 만들 수 있습니다.

응답 예 (개행으로 종료):

```
{"ok":true,"dist_cm":45,"yaw":1.2,"batt":7.8,"state":"OK"}\n
```

필드:

ok	— 마지막 명령 수락 여부
dist_cm	— 전방 거리 센서 측정값 (cm)
yaw	— IMU 자이로 yaw 값 (도)
batt	— 배터리 전압 (V)
state	— TC237 내부 상태 ("OK", "OBSTACLE", "LOW_BATT")

이 응답으로 라즈베리파이는 — TC237 에서 측정한 거리, 자세, 배터리 상태를 한꺼번에 받습니다. 별도 센서를 라즈베리파이에 붙일 필요가 없습니다.

Python 송수신 코드

```
import serial, json, time

class Tc237Link:
    def __init__(self, port="/dev/ttyUSB0", baudrate=115200):
        self.ser = serial.Serial(port, baudrate, timeout=0.1)
```

```

self.STX, self.ETX = 0x02, 0x03

def send(self, cmd, p1=0, p2=0):
    checksum = cmd ^ p1 ^ p2
    pkt = bytes([self.STX, cmd, p1, p2, checksum, self.ETX])
    self.ser.write(pkt)

def read_status(self, timeout=0.2):
    """JSON 한 줄을 받아서 dict 로 반환. 실패하면 None."""
    deadline = time.time() + timeout
    buf = b""
    while time.time() < deadline:
        chunk = self.ser.read(64)
        if chunk:
            buf += chunk
            if b"\n" in buf:
                line = buf.split(b"\n")[0].decode("ascii", errors="ignore")
                try:
                    return json.loads(line)
                except json.JSONDecodeError:
                    return None
    return None

# 편의 메서드
def forward(self, speed):
    self.send(0x01, max(0, min(100, speed)))
def stop(self):
    self.send(0x02)
def steer(self, strength):
    if strength < 0:
        self.send(0x03, abs(strength), 0) # 좌
    else:
        self.send(0x03, strength, 1)      # 우
def emergency(self):
    self.send(0x06)

```

안전의 마지막 방어선

이 프로토콜의 중요한 특징 — TC237 은 라즈베리파이의 명령을 그대로 따르지 않습니다. 거리 센서로 30cm 안에 장애물이 있으면 "FORWARD" 명령을 받아도

무시하고 정지합니다. AI가 만에 하나 잘못 판단해도 — 결정의 마지막 책임이 안전한 MCU에 있습니다.

기능안전의 원칙:

이건 "두 단계 안전 검증"의 패턴입니다. 자동차의 AEB(Autonomous Emergency Braking) 시스템도 같은 원리입니다 — 카메라/레이더가 충돌 가능성을 판단하고, MCU가 그 판단을 한 번 더 검증한 뒤 브레이크를 작동합니다. 단일 실패점(single point of failure)을 만들지 않는 — 안전 시스템 설계의 기본입니다.

8.7 AI와 함께하는 통합 시스템 실습

Vol.4의 마지막 실습입니다. 모든 조각을 하나의 프로그램으로 합칩니다. 매우 큰 코드가 되므로 — AI와 함께 단계적으로 만들어 갑니다.

1 단계 — 골격 만들기

AI와 함께 설계하기

다음 요소들을 통합한 Python 스크립트의 골격을 짜 줘. 클래스 분리만 잘 하고 실제 로직은 비워둬: 1) Camera 클래스 — picamera2 캡처 2) Detector 클래스 — YOLOv8(ONNX) 추론 3) Tc237Link 클래스 — 8.6의 UART 프로토콜 4) GpsReader 클래스 — 7장의 GPS NMEA 파싱 5) TrackingFSM 클래스 — 8.4의 상태 머신 6) Steering 클래스 — 8.2의 PD 제어 7) Logger 클래스 — 8.5의 CSV 로깅 8) Main 함수 — 위 객체들을 조립하고 메인 루프 실행 각 클래스의 `__init__` 시그니처와 메서드 이름만 정확히 — 본문은 pass로.

이렇게 골격부터 시작하면 — "어디까지가 누구의 책임인가"를 명확히 한 뒤 코드를 채워갈 수 있습니다.

2 단계 — 각 클래스 채우기

AI에게 한 번에 전체를 부탁하기보다, 클래스 하나씩 부탁하면 검증하기 좋습니다.

AI와 함께 설계하기

위 골격에서 Detector 클래스만 채워 줘. 요구사항: 1) 생성자에 ONNX 모델 경로와 신뢰도 임계값(기본 0.5) 2) `detect(frame)` 메서드 — numpy 이미지 받아서 [(class_id, conf, x1, y1, x2, y2), ...] 리스트 반환 3) 단일 클래스 추적 가정 — 가장 신뢰도 높은 박스 하나만 반환하는 `detect_best(frame)` 메서드도 같이 4) 추론 실패

시 빈 리스트 반환 (예외 안 던짐) ONNX 런타임은 onnxruntime 사용. 라즈베리파이 4에서 잘 동작해야 함.

같은 패턴으로 Camera, Tc237Link, GpsReader, TrackingFSM, Steering, Logger를 차례로 만듭니다. 매번 AI에게 "이 클래스만 — 위에 정의한 인터페이스를 따라"라고 부탁드립니다.

3 단계 — 메인 루프 통합

AI와 함께 설계하기

이제 메인 루프를 작성해 줘. 30 FPS 목표. 매 프레임: 1) 카메라에서 프레임 받기 2) Detector로 고양이 검출 3) Detector 결과를 TrackingFSM에 전달, 다음 상태 결정 4) FSM이 'TRACKING' 상태면 Steering으로 PD 계산 5) 결정된 명령을 Tc237Link로 송신 6) TC237 응답(거리, 배터리)을 받아 다음 사이클에 반영 7) GpsReader는 별도 스레드에서 갱신 (메인 루프는 읽기만) 8) Logger에 한 줄 기록 9) 키보드 'q'로 종료 시 모든 직렬 포트와 카메라를 안전하게 닫기 예외 처리도 꼼꼼히 — 어떤 한 모듈이 실패해도 전체가 죽지 않도록.

4 단계 — 검증과 디버깅

동작이 시작되면 — 한 번에 모든 것을 켜지 말고, 단계적으로 검증합니다.

- 스텝 1 — 모터 끄고 코드만 실행. 검출·FSM·명령 결정이 화면에서만 동작하는지 확인.
- 스텝 2 — RC 카를 손에 들고 카메라 앞에 고양이 사진을 보여줌. 모터가 어느 방향으로 도는지 확인.
- 스텝 3 — 바닥에 놓고 짧은 거리에서 테스트. 안전을 위해 손으로 잡을 준비.
- 스텝 4 — 마당이나 공원에서 실제 고양이로 테스트.

이 "단계적 검증"은 시간이 더 걸리는 것 같지만 — 사실은 빠른 길입니다. 한 번에 다 켜면 무엇이 잘못된 것인지 구분하기 어려워 디버깅이 늘어집니다.

AI와 함께 설계하기

위 통합 시스템에서 실외 테스트 전에 점검해야 할 사항을 체크리스트로 만들어 줘. 카테고리: 1) 하드웨어 — 결선, 전원, 케이블 고정 2) 소프트웨어 — 의존성, 권한, 자동 실행 3) 안전 — 비상 정지, 영역 제한, 배터리 모니터 4) 환경 — 날씨, 장소, 사람·동물

안전 5) 데이터 — 로그, GPS fix, 모델 파일 각 항목은 한 줄로 — 실제로 RC 카 옆에 두고 "체크/체크" 할 수 있는 형식.

8.8 Vol.4 를 마치며 — 우리가 만든 것의 의미

축하합니다. 우리는 Physical AI 의 작은 한 조각을 직접 만들었습니다.

이게 무슨 의미가 있을까요? 표면적으로는 "고양이를 따라다니는 RC 카"입니다. 그러나 본질적으로 우리가 만든 것은 — 카메라로 세상을 보고, AI 로 의미를 추출하고, 결정을 내려, 모터로 행동하는 시스템입니다. 인공지능이 화면 안에 갇혀 있던 시대를 끝내고, 바퀴를 달고 세상으로 나오는 — 그것이 Physical AI 입니다.

우리 RC 카와 자율주행차의 거리

실제로 자율주행차, 산업용 로봇, 농업 드론, 가정용 로봇청소기까지 — 우리 시대의 거의 모든 "움직이는 AI 시스템"이 똑같은 구조로 동작합니다.

Tesla / Waymo / 산업용 로봇 / 우리 RC 카

[인지]	→ 카메라, LiDAR (또는 카메라만)
[인식 AI]	→ 객체 검출, 차선 검출, 깊이 추정
[의사결정]	→ 경로 계획, 상태 머신
[안전 검증]	→ 다중 센서, 안전 MCU
[액추에이터]	→ 모터, 서보, 스티어링

이 구조는 시스템 크기에 따라 부품의 가격과 정교함이 다를 뿐 — 원리는 다 같습니다.

우리가 만든 RC 카는 같은 원리의 가장 단순한 시제품입니다. 그러나 "단순한 시제품"을 만들 줄 아는 사람이 "복잡한 양산품"을 만드는 길에 들어선 사람입니다.

AI 와 함께 — 그러나 책임은 우리의 손에

그리고 우리는 이 모든 과정에서 AI 와 함께 작업했습니다.

- 부품을 고를 때 — AI 에게 트레이드오프를 물었습니다.
- 아키텍처를 그릴 때 — AI 에게 검토를 부탁했습니다.

- 코드를 짤 때 — AI 와 짝코딩했습니다.
- 결과를 해석할 때 — AI 에게 자문을 받았습니다.

그러나 결정은 우리가 했고, AI 는 좋은 후배 엔지니어였습니다. AI 는 빠르게 초안을 만들고 다양한 옵션을 제시했지만 — "이게 우리 환경에서 맞는가"는 우리가 판단해야 했습니다. 데이터시트를 펴 보고, 실측하고, 검증하고, 결정했습니다. 이것이 AI 시대 임베디드 개발자의 일하는 방식입니다. AI 를 쓰지 않는 개발자는 점점 느려지고, AI 에 의존하기만 하는 개발자는 검증 없이 무너집니다. 두 함정 사이의 균형 위에서 — 좋은 시스템이 만들어집니다.

시리즈 4 권의 여정

Vol.1 부터 **Vol.4** 까지의 여정에서 마지막으로 드리고 싶은 말씀이 있습니다.

Vol.1 에서 우리는 TC237 보드를 처음 만났습니다 — "GPIO 한 핀을 어떻게 켜는가"를 배웠습니다.

Vol.2 에서 페리페럴을 정복했습니다 — ADC, PWM, UART, SPI, CAN, Flash.

Vol.3 에서 RC 카를 만들었습니다 — 모터, 무선 통신, 거리 센서, 자이로 보정.

Vol.4 에서 AI 를 엮었습니다 — 카메라, YOLO, 파인튜닝, GPS, Physical AI 통합.

4 권 사이의 거리는 멀니다. "GPIO 한 핀"에서 "신경망과 GPS 를 통합한 자율 시스템"까지 — 그러나 발자국 한 걸음 한 걸음은 가까웠습니다. 매 권마다 직전 권의 지식 위에 한 층씩 쌓아 올렸습니다. 이것이 시리즈로 책을 쓴 이유입니다.

임베디드 개발자는 더 이상 회로도만 데이터시트만의 세계에 있지 않습니다. 우리의 코드는 이제 — 카메라 픽셀에서 신경망 가중치로, 명령 프로토콜에서 모터 PWM 으로 — 하나의 거대한 흐름의 일부입니다. 그 흐름을 끝에서 끝까지 이해하고 만들 수 있는 사람이 이 시대의 임베디드 엔지니어입니다.

AI 와 함께, 그러나 책임은 우리의 손에 — 그 균형 위에 좋은 시스템이 만들어집니다.

이 책을 끝까지 읽어주셔서 감사합니다.

— Toby

Chapter 8 핵심 정리

라즈베리파이 = 인지/판단, TC237 = 안전한 실행 — 책임의 분리.

화면 좌표 → 조향 명령 변환은 차선 유지(LKA)와 같은 비례 제어 원리.

P 제어는 부드러움, PD 제어는 응답성. PID 까지 안 가도 RC 카에 충분.

단일 카메라 거리 추정은 "알고 있는 크기 / 박스 크기" — 정면 자세 가정.

추적은 상태 머신으로 — SEARCH/TRACKING/LOST/STOPPED/IDLE.

TC237 은 라즈베리파이 명령을 검증한 뒤 실행 — AI 가 틀려도 차가 부서지지 않는다.

통합 코드는 클래스 단위로 단계적으로 만든다. AI 에게 한 번에 다 부탁하지 말 것.

Physical AI = 카메라 + AI + 컨트롤러 + 액추에이터. 이 구조는 자율주행·로봇·드론과 같다.

부록

이 부록은 Vol.4의 실습을 따라할 때 참고할 수 있는 보조 자료들을 모아 두었습니다. 본문이 "왜"와 "어떻게"에 집중했다면, 부록은 "무엇을"과 "어디서"에 답합니다 — 어떤 부품을 사고, 어떻게 연결하고, 문제가 생기면 어떻게 해결하는지.

A.1 추천 부품 리스트와 예상 비용

Vol.4 RC 카를 처음부터 만들 때 필요한 부품 리스트입니다. Vol.3에서 이미 가지고 있는 부품(TC237 보드, 모터 드라이버, IR 센서 등)은 제외했습니다.

필수 부품

번호	품목	추천 모델	예상 가격(KRW)
1	라즈베리파이	Pi 4 Model B 4GB	약 80,000
2	microSD 카드	32GB Class 10	약 12,000
3	라즈베리파이 전원	5V 3A USB-C 어댑터	약 15,000
4	라즈베리파이 카메라	Pi Camera Module 3	약 35,000
5	카메라 리본 케이블	15cm or 50cm	약 5,000
6	GPS 모듈	NEO-M8N + 안테나	약 20,000
7	USB-UART 어댑터	FT232 또는 CP2102	약 8,000
8	점퍼 와이어	M-F, F-F 각 20 개	약 5,000
9	라즈베리파이 케이스+팬	능동 쿨링 권장	약 15,000
소계			약 195,000

선택 부품

번호	품목	용도	예상 가격(KRW)
10	Pi 5 (대신)	추론 FPS 2~3 배	+50,000
11	Coral USB Accelerator	TPU 가속, FPS 4~5 배	약 80,000
12	HDMI 마이크로 케이블	초기 셋업용	약 5,000
13	HQ Camera Module	고화질 (M-mount 렌즈)	약 70,000
14	스테레오 카메라 모듈	깊이 추정	약 50,000

15 ToF 센서 (VL53L1X) cm 급 거리 측정 약 15,000

최소 구성(필수만)으로 약 20 만원, Pi 5 와 가속기까지 더하면 33 만원 전후입니다.
이미 라즈베리파이를 가지고 있다면 추가 비용은 8~10 만원 수준입니다.

A.2 라즈베리파이 GPIO 핀맵과 RC 카 결선도

Vol.4 RC 카 통합 시스템의 전체 결선을 정리합니다. 라즈베리파이 측만 다룹니다 —
TC237 측 결선은 Vol.3 부록을 참조하세요.

라즈베리파이 핀 사용 현황

Pi GPIO 핀맵 (필수 사용 핀만)

핀번호	GPIO	기능	연결 대상
1	3.3V	전원	(사용 안 함)
2	5V	전원	GPS VCC, USB-UART VCC
4	5V	전원	여분
6	GND	공통접지	GPS GND, USB-UART GND
8	GPIO14	UART TX	(GPS 는 단방향이라 미사용)
10	GPIO15	UART RX	GPS TX

주의: GPIO14, 15 는 disable-bt 설정 후 사용 가능.

USB 포트:

Port 1 (USB 2.0) ← USB-UART 어댑터 (TC237 연결)

Port 2 (USB 2.0) ← 키보드 (셋업 시에만)

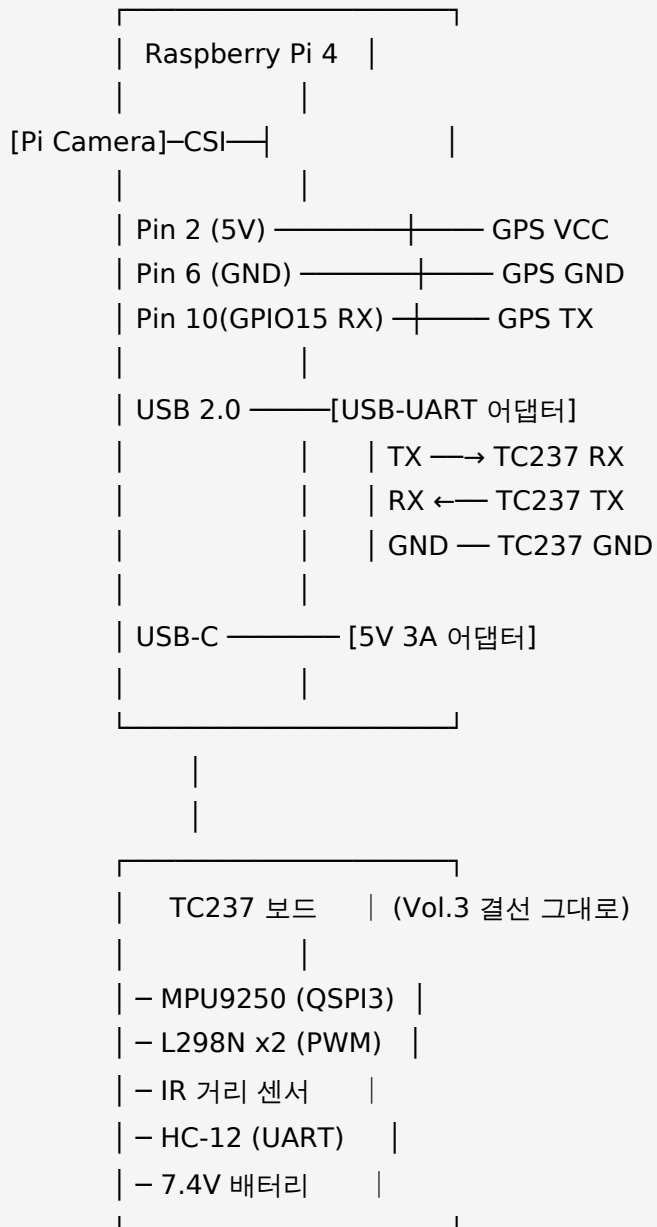
Port 3 (USB 3.0) ← USB 카메라 (사용 시)

Port 4 (USB 3.0) ← 외장 SSD (옵션)

CSI 포트:

Camera 0 ← Pi Camera 리본 케이블

전체 결선도



배선 시 주의사항

- Pi 전원과 RC 카 배터리는 분리 — Pi 는 자체 5V 어댑터(또는 별도 모바일 배터리), RC 카는 7.4V 배터리. 공통 GND 는 USB-UART 의 GND 한 핀으로 연결됩니다.
- USB-UART 어댑터의 3.3V/5V 점퍼 확인 — TC237 UART 는 3.3V 레벨. 5V 로 설정된 어댑터를 직결하면 TC237 이 손상될 수 있습니다.
- GPS 안테나 방향 — 평평한 면이 하늘을 향해야 합니다. RC 카 위에 장착할 때 카메라나 모터에 가려지지 않게 유의.

- 케이블 고정 — RC 카가 움직이며 케이블이 빠지는 게 가장 흔한 문제.
글루건이나 케이블 타이로 고정 권장.

A.3 자주 발생하는 문제와 해결 (Troubleshooting)

Vol.4 실습을 진행하면서 만날 가능성이 높은 문제들을 모아두었습니다. 막힐 때 이 절을 먼저 확인하세요.

카메라 관련

증상: "camera0 not found" 또는 picamera2 import 실패

원인 후보:

- 1) 카메라 인터페이스 비활성화
- 2) Bookworm 이전 OS (구버전 picamera 사용 중)
- 3) 리본 케이블 접촉 불량

해결:

```
sudo raspi-config → Interface Options → Camera → Enable  
sudo apt install -y python3-picamera2 python3-libcamera  
케이블 양쪽을 다시 꽂고 (금속 면 방향 주의), 재부팅
```

증상: 카메라 영상이 너무 어둡거나 색이 이상함

원인 후보:

- 1) 화이트 밸런스 자동 설정 비활성
- 2) 노출(exposure) 시간 설정 부적절
- 3) 케이블 노이즈로 인한 색상 깨짐

해결:

```
picam2.set_controls({"AwbEnable": True, "AeEnable": True})  
케이블 길이가 50cm 넘으면 신호 감쇠. 짧은 케이블 사용.
```

YOLO/추론 관련

증상: YOLO 추론이 너무 느림 (1~2 FPS)

원인 후보:

- 1) PyTorch 런타임의 오버헤드
- 2) 입력 해상도가 너무 큼
- 3) Pi 4 에서 큰 모델 사용 (yolov8s 이상)

해결:

- 1) ONNX 변환: `model.export(format="onnx")`
- 2) INT8 양자화 추가
- 3) 입력 해상도 640 → 416 또는 320
- 4) yolov8n 사용 확인 (s/m/l/x 는 라즈베리파이에 부담)
- 5) Pi 5 로 업그레이드 또는 Coral USB 추가

증상: 학습은 잘 됐는데 라즈베리파이에서 인식이 잘 안 됨

원인 후보:

- 1) 라즈베리파이 카메라 vs 학습 데이터의 도메인 차이
- 2) 양자화로 정확도 손실
- 3) 신뢰도 임계값이 너무 높음

해결:

- 1) 학습 데이터를 라즈베리파이 카메라로 직접 찍어 보강
- 2) FP16 양자화 시도 (INT8 대신)
- 3) confidence threshold 0.5 → 0.3 으로 낮춰서 테스트

GPS 관련

증상: GPS fix 가 아예 안 잡힘 (수신기 LED 안 깜빡임)

원인 후보:

- 1) 실내 또는 차량 안 (신호 약함)
- 2) 안테나가 뒤집혀 있음
- 3) UART 설정 오류 (Bluetooth 점유)
- 4) 모듈 전원/접지 불량

해결:

- 1) 야외, 하늘이 트인 곳에서 5 분 이상 대기 (콜드 스타트)
- 2) 안테나의 평평한 면을 하늘로
- 3) /boot/firmware/config.txt 에 disable-bt 추가 후 재부팅
- 4) 5V/3.3V 핀 확인, GND 견고히 연결

증상: GPS 데이터가 들어오는데 위치가 자꾸 튕 (다중경로)

원인 후보:

- 1) 빌딩 사이, 큰 차량 옆 (신호 반사)
- 2) 위성 수가 4 개 미만
- 3) HDOP 값이 큼 (≥ 5)

해결:

- 더 트인 장소로 이동
- HDOP > 5 인 데이터는 사용하지 않도록 필터링
- 이동 평균(window 5~10) 적용으로 노이즈 완화

UART 통신 관련

증상: TC237 에 명령을 보내도 응답이 없음

원인 후보:

- 1) TX/RX 크로스 결선 안 됨
- 2) 보레이트 불일치
- 3) USB-UART 어댑터 드라이버 미설치
- 4) 권한 문제

해결:

- 1) Pi TX → TC237 RX, Pi RX → TC237 TX (크로스 확인)
- 2) 양쪽 보레이트 동일 (예: 115200)
- 3) ls /dev/ttyUSB*로 어댑터 인식 확인
- 4) sudo usermod -aG dialout \$USER 후 재로그인

전원 관련

증상: Pi가 부팅 도중 재시작되거나 화면에 번개 모양

원인:

공급 전류 부족. Pi 4는 최대 3A, Pi 5는 5A 필요.

해결:

공식 5V 3A 어댑터 사용. 모바일 보조배터리는 PD 지원 25,000mAh 이상 권장.

카메라, GPS, Coral 모두 연결 시 전류가 4A 가까이 갈 수 있음.

A.4 학습 데이터·코드 다운로드 안내

Vol.4의 모든 예제 코드, 학습 스크립트, 샘플 데이터셋은 다음에서 받을 수 있습니다.

GitHub 저장소

<https://github.com/aicraftlog/embedded-ai-vol4>

폴더 구조:

```
embedded-ai-vol4/
├── ch02_setup/
│   ├── setup_pi.sh
│   └── verify_install.py
├── ch03_color_detect/
│   ├── hsv_calibration.py
│   └── blue_ball_tracker.py
├── ch04_yolo_inference/
│   ├── yolo_basic.py
│   └── cat_detector.py
├── ch05_auto_label/
│   ├── extract_frames.py
│   ├── auto_label.py
│   └── dataset_split.py
├── ch06_finetune/
│   ├── train.py
│   ├── evaluate.py
│   └── export_onnx.py
└── ch07_gps/
```

```
| ├── gps_basic.py
| ├── gps_logger.py
| └── gps-logger.service
└── ch08_integration/
    ├── main.py
    ├── camera.py
    ├── detector.py
    ├── tc237_link.py
    ├── gps_reader.py
    ├── tracking_fsm.py
    ├── steering.py
    └── logger.py
└── README.md
```

샘플 데이터셋

저자 본인의 고양이 사진 300 장 + 자동 라벨링 결과로 만든 샘플 데이터셋입니다.

<https://aircraftlog.com/downloads/vol4-dataset>

포함된 것:

- images/train/ (240 장)
- images/val/ (60 장)
- labels/train/ (240 개 .txt)
- labels/val/ (60 개 .txt)
- data.yaml
- sample_video.mp4 (직접 추출해보고 싶을 때)

용도: 학습 과정을 빨리 따라해 보기 위함.

실제 "여러분의 고양이"를 위해서는 여러분의 데이터로 다시 학습 필요.

학습된 샘플 모델

같은 데이터로 학습한 best.onnx, best_int8.onnx 파일도 함께 제공합니다. "

학습은 다음에 하고 일단 추론부터 돌려보고 싶다"는 분께 도움이 됩니다.

A.5 더 깊이 공부하고 싶다면 — 추천 리소스

Vol.4 를 마쳤다면 이제 더 넓은 세계로 나갈 준비가 된 것입니다. 다음 단계 학습을 위한 추천 자료를 분야별로 정리했습니다.

컴퓨터 비전 / 딥러닝 기초

- CS231n (Stanford) — 컴퓨터 비전 입문 강의. YouTube 무료.
- PyTorch 공식 튜토리얼 — "60 분 만에 PyTorch 입문"부터 시작.
- Andrej Karpathy 의 "Zero to Hero" YouTube 시리즈 — 신경망을 처음부터 직접 만들어가는 강의.

객체 검출 / YOLO 심화

- Ultralytics 공식 문서 — YOLOv8/v9/v10 사용법, 학습 팁.
- YOLO 논문들 — v1 (2015) → v8 (2023) 까지의 진화. "YOLOv1" 원논문은 처음 읽기에 가장 좋음.

라즈베리파이 + AI 실전

- Raspberry Pi 공식 "AI Kit" 문서 — Hailo NPU 활용법.
- Coral.ai 공식 예제 — TPU 가속의 모든 것.
- Edge Impulse — 임베디드용 ML 모델 학습 SaaS (무료 티어 있음).

자율주행 / 로봇

- ROS 2 공식 튜토리얼 — 로봇 SW 의 표준 프레임워크.
- Apollo (Baidu 의 오픈소스 자율주행) — 실제 양산 시스템의 구조를 볼 수 있음.
- Sebastian Thrun 의 "Probabilistic Robotics" — 자율주행의 수학적 기반.

AI 협업 / 프롬프트 엔지니어링

- Anthropic 의 공식 문서 — Claude 를 효과적으로 사용하는 법.
- GitHub Copilot Best Practices — 실제 코딩 보조 도구 활용 사례.

A.6 시리즈 전체 회고 — Vol.1 부터 Vol.4 까지

이 시리즈를 처음 기획할 때 한 가지 약속을 했습니다 — "임베디드 개발자가 AI 시대를 살아갈 수 있도록 길을 깔아주는 책"이 되겠다는 약속이었습니다.

4 권의 여정을 한눈에

Vol.1 — 임베디드의 기초

TC237 보드 처음 만지기

GPIO 한 핀을 켜고 끄는 일부터

AI 와 함께 코드를 처음 부탁해보기

Vol.2 — MCU 페리페럴

ADC, PWM, UART, SPI, CAN, Flash

MCU 의 손과 발을 차례로 정복

AI 도움으로 페리페럴 드라이버 작성

Vol.3 — RC 카 제작

모터, 무선 통신, 거리 센서, IMU

단일 MCU 로 만드는 자율 시스템의 상한선

실제 자동차 기술(XCP, ESP, UDS)과의 평행 비유

Vol.4 — Physical AI

라즈베리파이 + YOLOv8 + GPS + TC237

카메라로 보고, AI 로 판단하고, MCU 로 안전하게 실행

AI 는 코드 도구를 넘어 시스템 설계의 동료로

Vol.1 에서 "GPIO 한 핀"을 처음 만지셨던 분이, Vol.4 에서는 "신경망 추론 결과를 UART 로 보내 RC 카를 자율 주행시키는" 코드를 짜고 있습니다. 4 권 사이의 거리는 멀니다 — 그러나 발자국 한 걸음 한 걸음은 가까웠습니다. 이것이 시리즈로 책을 쓴 이유입니다.

이 시리즈가 끝나는 자리에서

이 책은 끝났지만, 여러분의 프로젝트는 이제 시작입니다.

- RC 카를 더 빠르게 — Pi 5 로 업그레이드, Coral 가속기 추가, NCNN 런타임 검토.
- 카메라를 더 정확하게 — 더 많은 데이터, 더 다양한 자세, 새로운 객체 추가.
- 모델을 더 가볍게 — TensorRT 변환, pruning, knowledge distillation 도전.
- 새로운 임무 — 사람 추적, 특정 색깔 공 추적, 줄 따라가기 등.
- Vol.5 — Vol.4 가 완결편이지만, 여러분만의 "Vol.5"를 만드세요. 그것이 진짜

학습입니다.

여러분이 다음에 무엇을 만드실지 정말 궁금합니다. AI-Craft 유튜브 채널과 블로그 (aiaircraftlog.com)에서, 그리고 여러분이 만드신 프로젝트를 공유해 주시는 자리에서, 다시 만나길 기대합니다.

긴 여정 함께 해주셔서, 진심으로 감사합니다.

— Toby
AI-Craft